

Índice

1	Plantillas Base y Optimización I/O	1
1.1	Configuración Inicial (Template C++)	1
1.2	Lectura Eficiente (Fast I/O)	1
2	Casteos y Conversiones	2
2.1	Binario a Decimal	2
2.2	Decimal a Binario	2
2.3	String a Entero	2
2.4	Entero a String	2
2.5	String a Long Long	2
2.6	Char a Entero	2
2.7	Entero a Char	2
2.8	Char a Mayúscula y Minúscula	2
2.9	String a Mayúsculas	2
2.10	String a Minúsculas	2
2.11	Decimal a Hexadecimal	3
2.12	Hexadecimal a Decimal	3
2.13	Decimal a Cualquier Base	3
2.14	Cualquier Base a Decimal	3
2.15	String a <code>double</code>	3
2.16	Entero a String con Formato	3
2.17	ASCII: Char a Código	3
2.18	Código ASCII a Char	3
2.19	String Numérico a Vector de Dígitos	3
2.20	Vector de Dígitos a String	4
3	Estructuras de Datos y Búsqueda	5
3.1	Búsqueda Binaria (Algoritmo Floor)	5
3.2	Fenwick Tree (Binary Indexed Tree)	5
3.3	Segment Tree (Consultas por Rangos)	5
3.4	Segment Tree con Lazy Propagation	6
3.5	Merge Sort Tree	6
3.6	Sparse Table (RMQ Estático)	6
3.7	DSU con Unión por Tamaño (Rollback-Ready)	7
3.8	Trie (Árbol de Prefijos)	7
4	Lower Bound y Upper Bound	9
4.1	Conceptos y Uso Básico en STL	9
4.2	Búsqueda con Comparador Personalizado	9
4.3	<code>lower_bound</code> / <code>upper_bound</code> en <code>set</code> y <code>multiset</code>	9
4.4	Búsqueda Binaria Genérica sobre Predicado (Adaptativa)	10
4.5	Implementación Manual (Referencia Interna)	10
5	Algoritmos de Grafos	11
5.1	Tabla de Decisión Rápida	11
5.2	Plantilla Universal de Grafo	11
5.3	BFS (Búsqueda en Anchura)	11
5.3.1	BFS Multi-Fuente	12
5.3.2	BFS 0/1 (Deque)	12
5.4	DFS (Búsqueda en Profundidad)	12
5.4.1	DFS Iterativo (evita stack overflow)	12
5.5	Componentes Conexas y Grafos Bipartitos	13
5.6	Detección de Ciclos	13
5.6.1	Grafo NO Dirigido	13
5.6.2	Grafo Dirigido (3 colores)	13
5.7	Ordenamiento Topológico	13

5.7.1	Kahn (BFS) — Recomendado	13
5.7.2	DFS (post-orden invertido)	14
5.8	Caminos Mínimos con Pesos	14
5.8.1	Dijkstra (pesos no negativos)	14
5.8.2	Bellman-Ford (permite pesos negativos)	14
5.8.3	Floyd-Warshall (todos los pares)	14
5.9	Árbol de Expansión Mínima (MST)	15
5.9.1	Kruskal (recomendado, más fácil de codear)	15
5.9.2	Prim (mejor para grafos densos)	15
5.10	Componentes Fuertemente Conexas: Tarjan	15
5.11	Puentes y Puntos de Articulación	15
5.12	Ancestro Común Más Bajo (LCA) — Binary Lifting	16
5.13	2-SAT	16
5.14	Flujo Máximo: Edmonds-Karp	16
5.15	Grafos Implícitos en Grid/Matriz	17
5.16	Errores Comunes que Cuestan Penalizaciones	17
6	Cadenas (Strings)	18
6.1	KMP (Knuth-Morris-Pratt)	18
6.2	Z-Function	18
6.3	Hashing de Strings (Polinomial)	18
6.4	Manacher (Palíndromos)	18
6.5	Boyer-Moore (Búsqueda de Patrones)	19
7	Programación Dinámica (DP)	20
7.1	¿Cuándo Usar Programación Dinámica?	20
7.2	Plantilla General de DP (Top-Down / Memoización)	20
7.3	Fibonacci con Memoización	20
7.3.1	Versión con Arreglo	20
7.3.2	Versión con <code>unordered_map</code>	21
7.3.3	Fast Doubling con Memoización	21
7.4	Mochila 0-1 (0-1 Knapsack)	21
7.5	Longest Common Subsequence (LCS)	21
7.6	Longest Increasing Subsequence (LIS)	21
7.7	DP con Máscaras de Bits (Bitmask DP - TSP)	22
7.8	DP en Árboles (Tree DP)	22
8	Manipulación de Bits (Bitwise)	23
8.1	Operaciones Básicas y Trucos Fundamentales	23
8.2	Conteo de Bits (Popcount)	23
8.3	Suma y Resta Binaria (Bit a Bit)	23
8.4	Multiplicación Binaria (Russian Peasant / Egyptian Multiplication)	24
8.5	Enumeración de Submáscaras (Submask Enumeration)	24
8.6	Propiedades y Trucos con XOR	24
8.7	<code>std::bitset</code> : Uso y Conversión	25
8.8	Casos de Uso de Bitmasking	25
9	Matemáticas Teóricas	27
9.1	MCD (Máximo Común Divisor) y MCM (Mínimo Común Múltiplo)	27
9.2	Algoritmo de Euclides Extendido	27
9.3	Criba de Eratóstenes (Sieve)	27
9.4	Exponenciación Rápida (Fast/Modular Power)	27
9.5	Inverso Multiplicativo Modular	28
9.6	Combinatoria: Factoriales y nCr con Módulo	28
9.7	Test de Primalidad Miller-Rabin	28
9.8	Teorema Chino del Resto (CRT) — Implementación	28
9.9	Función de Möbius	29
9.10	Fórmula de Legendre (Exponente de un Primo en $N!$)	29

10 Geometría Computacional	30
10.1 Primitivas Básicas: Punto, Producto Cruz y Punto	30
10.2 Envolverte Convexa (Convex Hull - Andrew Monotone Chain)	30
10.3 Área y Punto Dentro de Polígono	30
10.4 Intersección de Segmentos	30
10.5 Punto de Intersección de Dos Rectas	31
10.6 Distancia Punto-Recta y Punto-Segmento	31
10.7 Círculos: Pertenencia e Intersección	31
10.8 Par de Puntos Más Cercano (Closest Pair)	32
10.9 Rotating Calipers: Diámetro de un Conjunto de Puntos	32
10.10 Ordenamiento Angular (Radial Sort) y Convexidad	32
10.11 Interpolación de Hermite (Cúbica)	33
10.12 Spline Cúbico Natural	33
11 Fórmulas Matemáticas Esenciales (Implementadas)	35
11.1 Sumas y Series	35
11.2 Combinatoria	35
11.3 Geometría	36
11.4 Probabilidad y Recurrencias	36
11.5 Teorema Maestro (guía de referencia, no se codea)	37
12 Casos de Estudio: Problemas Resueltos	38
12.1 Matrioshka	38

EroMergeSort Team Notebook

Manual de Referencia Completo (C++)

14 de agosto de 2026

1. Plantillas Base y Optimización I/O

1.1 Configuración Inicial (Template C++)

Estructura inicial limpia con macros habituales para acelerar la escritura de código durante los concursos.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 #define fastio ios_base::sync_with_stdio(0); cin.tie(0);
4   cout.tie(0);
5 #define pb push_back
6 #define all(v) v.begin(), v.end()
7 typedef long long ll;
8 typedef pair<int, int> pii;
9 int main() {
10     fastio;
11     // Tu código aquí
12     return 0;
13 }
```

Ejemplo de uso

Usa esta plantilla cuando empieces un problema de concurso: activa I/O rápido y deja un punto claro para la solución.

```
1 int main() {
2     fastio;
3     int n;
4     cin >> n;
5     // Resolver el problema...
6     return 0;
7 }
```

1.2 Lectura Eficiente (Fast I/O)

Complejidad de Tiempo: $\mathcal{O}(1)$ por operación de lectura amortizada.

Desactiva la sincronización pesada de flujos heredada de C y desvincula cin de cout para agilizar lecturas masivas. ****No mezclar con scanf/printf ni usar endl.****

```
1 // Incluir inmediatamente al inicio del main
2 ios_base::sync_with_stdio(false);
3 cin.tie(NULL);
```

Ejemplo de uso

Úsalo cuando el juez entrega cientos de miles o millones de datos.

```
1 int n;
2 cin >> n;
3 vector<int> a(n);
4 for (int &x : a) cin >> x;
5 cout << a[0] << '\n';
```

Patrones de Entrada Comunes

```
1 void lectura_ejemplos() {
2     int n; double d; char c;
3     cin >> n >> d >> c; // Variables basicas
4     vector<int> arr(n);
5     for (int i = 0; i < n; i++) cin >> arr[i]; // Carga de
6     vectores
7     string nombre;
8     cin.ignore(); // Limpiar salto de linea remanente del
9     bufer
10    getline(cin, nombre); // Lineas completas con espacios
11    int x;
```

```
10 while (cin >> x) { /* Leer hasta Fin de Archivo (EOF)
11    */ }
```

Ejemplo de uso

Lee datos de entrada de forma cómoda y rápida.

```
1 int n; cin >> n;
2 vector<int> a(n);
3 for (int &x : a) cin >> x;
```

2. Casteos y Conversiones

Esta sección reúne conversiones frecuentes en programación competitiva. Cada ejemplo es autocontenido y muestra tanto la conversión como su uso práctico.

2.1 Binario a Decimal

La forma más sencilla es recorrer los bits como caracteres. También podemos usar `bitset` cuando el tamaño sea fijo.

```
1 string bin = "101101";
2 int decimal = stoi(bin, nullptr, 2);
3
4 cout << decimal << '\n';
5 // Salida: 45
```

2.2 Decimal a Binario

Podemos usar `bitset` para un tamaño fijo o construir la representación manualmente.

```
1 int n = 45;
2
3 string binario = bitset<8>(n).to_string();
4
5 cout << binario << '\n';
6 // Salida: 00101101
```

2.3 String a Entero

`stoi` convierte un `string` en `int`. Para valores grandes podemos usar `stoll`.

```
1 string s = "123456";
2
3 int x = stoi(s);
4
5 cout << x + 44 << '\n';
6 // Salida: 123500
```

2.4 Entero a String

`to_string` convierte un número a texto.

```
1 int x = 2026;
2
3 string s = to_string(x);
4
5 cout << "ICPC " + s << '\n';
6 // Salida: ICPC 2026
```

2.5 String a Long Long

Útil cuando el número puede superar el rango de un `int`.

```
1 string s = "1000000000000";
2
3 long long x = stoll(s);
4
5 cout << x << '\n';
6 // Salida: 1000000000000
```

2.6 Char a Entero

Para obtener el valor numérico de un dígito usamos la resta con `'0'`.

```
1 char c = '7';
2
3 int x = c - '0';
4
5 cout << x + 5 << '\n';
6 // Salida: 12
```

2.7 Entero a Char

Si tenemos un dígito entre 0 y 9 podemos convertirlo usando `'0'`.

```
1 int x = 7;
2
3 char c = '0' + x;
4
5 cout << c << '\n';
6 // Salida: 7
```

2.8 Char a Mayúscula y Minúscula

Las funciones de `cctype` permiten cambiar caracteres fácilmente.

```
1 char a = 'g';
2 char b = 'Q';
3
4 cout << (char)toupper(a) << '\n';
5 cout << (char)tolower(b) << '\n';
6
7 // Salida:
8 // G
9 // q
```

2.9 String a Mayúsculas

Útil para normalizar palabras antes de compararlas.

```
1 string s = "IcPc";
2
3 for (char &c : s)
4     c = toupper(c);
5
6 cout << s << '\n';
7 // Salida: ICPC
```

2.10 String a Minúsculas

La misma idea puede utilizarse para convertir todo el texto a minúsculas.

```
1 string s = "AlGoRiTmOs";
2
3 for (char &c : s)
4     c = tolower(c);
5
6 cout << s << '\n';
7 // Salida: algoritmos
```

2.11 Decimal a Hexadecimal

`stringstream` permite convertir fácilmente entre bases.

```
1 int n = 255;
2
3 stringstream ss;
4 ss << hex << n;
5
6 string hexadecimal = ss.str();
7
8 cout << hexadecimal << '\n';
9 // Salida: ff
```

2.12 Hexadecimal a Decimal

Indicamos base 16 al usar `stoi`.

```
1 string hex = "FF";
2
3 int decimal = stoi(hex, nullptr, 16);
4
5 cout << decimal << '\n';
6 // Salida: 255
```

2.13 Decimal a Cualquier Base

Para una base entre 2 y 36 podemos construir la representación manualmente.

```
1 string convertirBase(int n, int base) {
2     const string digitos =
3         "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ";
4
5     if (n == 0) return "0";
6
7     string ans;
8
9     while (n > 0) {
10         ans += digitos[n % base];
11         n /= base;
12     }
13
14     reverse(ans.begin(), ans.end());
15     return ans;
16 }
17
18 cout << convertirBase(45, 2) << '\n';
19 // Salida: 101101
```

2.14 Cualquier Base a Decimal

Podemos interpretar cada dígito con la fórmula posicional.

```
1 int valor(char c) {
2     if ('0' <= c && c <= '9')
3         return c - '0';
4
5     return c - 'A' + 10;
6 }
7
8 long long aDecimal(string s, int base) {
9     long long ans = 0;
10
11     for (char c : s)
12         ans = ans * base + valor(c);
13
14     return ans;
15 }
16
17 cout << aDecimal("101101", 2) << '\n';
18 // Salida: 45
```

2.15 String a double

Cuando necesitamos números con parte decimal podemos usar `stod`.

```
1 string s = "3.14159";
2
3 double x = stod(s);
4
5 cout << fixed << setprecision(2) << x << '\n';
6 // Salida: 3.14
```

2.16 Entero a String con Formato

Cuando necesitamos controlar el formato, `stringstream` es útil.

```
1 int hora = 7;
2 int minuto = 5;
3
4 stringstream ss;
5 ss << setfill('0') << setw(2) << hora << ':' <<
6     << setw(2) << minuto;
7
8 cout << ss.str() << '\n';
9 // Salida: 07:05
```

2.17 ASCII: Char a Código

Cada carácter tiene un valor numérico ASCII.

```
1 char c = 'A';
2
3 cout << (int)c << '\n';
4 // Salida: 65
```

2.18 Código ASCII a Char

También podemos hacer la conversión inversa.

```
1 int x = 65;
2
3 cout << (char)x << '\n';
4 // Salida: A
```

2.19 String Numérico a Vector de Dígitos

Muy útil para problemas donde necesitamos procesar cada dígito.

```
1 string s = "314159";
2
3 vector<int> digitos;
4
5 for (char c : s)
6     digitos.push_back(c - '0');
7
8 for (int x : digitos)
9     cout << x << ' ';
10
11 // Salida:
12 // 3 1 4 1 5 9
```

2.20 Vector de Dígitos a String

La operación inversa permite reconstruir el número como texto.

```
1 vector<int> digitos = {3, 1, 4, 1, 5, 9};
2
3 string s;
4
5 for (int x : digitos)
6     s += char('0' + x);
7
8 cout << s << '\n';
9 // Salida: 314159
```

3. Estructuras de Datos y Búsqueda

3.1 Búsqueda Binaria (Algoritmo Floor)

Complejidad de Tiempo: $\mathcal{O}(\log N)$

Complejidad de Espacio: $\mathcal{O}(1)$

Busca un valor en un rango ordenado. Si el elemento no existe, retorna de forma exacta el índice del mayor elemento inferior más cercano al buscado (*floor*). Retorna -1 si todos son mayores.

```
1 int binary_search_floor(const vector<int>& arr, int target) {
2     int low = 0, high = (int)arr.size() - 1;
3     int ans = -1;
4     while (low <= high) {
5         int mid = low + (high - low) / 2;
6         if (arr[mid] == target) return mid;
7         else if (arr[mid] < target) { ans = mid; low = mid
8             + 1; }
9         else high = mid - 1;
10    }
11    return ans;
}
```

Ejemplo de uso

Ejemplo: encontrar el precio más alto que no supera un presupuesto.

```
1 vector<int> precios = {10, 20, 35, 50, 80};
2 int presupuesto = 42;
3 int pos = binary_search_floor(precios, presupuesto);
4 // pos = 2, porque precios[2] = 35
5 cout << precios[pos];
```

3.2 Fenwick Tree (Binary Indexed Tree)

Complejidad de Tiempo:

- Inicialización: $\mathcal{O}(N)$
- Actualización puntual (Update): $\mathcal{O}(\log N)$
- Consulta por rango (Query): $\mathcal{O}(\log N)$

Complejidad de Espacio: $\mathcal{O}(N)$

Estructura eficiente para consultas de sumas acumuladas dinámicas y actualizaciones puntuales en un arreglo de tamaño N .

```
1 struct BIT {
2     int n; vector<int> tree;
3     BIT(int n) : n(n), tree(n + 1, 0) {}
4     void update(int i, int delta) {
5         for (; i <= n; i += i & -i) tree[i] += delta;
6     }
7     int query(int i) {
8         int sum = 0;
9         for (; i > 0; i -= i & -i) sum += tree[i];
10        return sum;
11    }
12    int query(int l, int r) { return query(r) - query(l - 1); }
13};
```

Ejemplo de uso

Ejemplo: guardar ventas diarias, actualizar un día y consultar la suma de un intervalo.

```
1 BIT bit(5);
2 for (int i = 1; i <= 5; i++) bit.update(i, i * 10);
3
4 bit.update(3, 25); // sumar 25 al día 3
5 cout << bit.query(2, 4); // suma de los días 2..4
```

3.3 Segment Tree (Consultas por Rangos)

Complejidad de Tiempo:

- Construcción: $\mathcal{O}(N)$
- Actualización puntual (Update): $\mathcal{O}(\log N)$
- Consulta por rango (Query): $\mathcal{O}(\log N)$

Complejidad de Espacio: $\mathcal{O}(N)$ (usa $2S$ nodos, donde S es la siguiente potencia de 2).

Estructura iterativa para consultas asociativas en intervalos dinámicos. Reduce memoria frente a la versión recursiva de $4N$ y evita llamadas recursivas. La consulta usa el intervalo semiabierto $[l, r)$.

```
1 // Versión iterativa: usa 2*size nodos y elimina el costo
2 // de recursión.
3 struct SegTree {
4     int n;
5     int size;
6     vector<int> tree;
7
8     explicit SegTree(int n) : n(n) {
9         size = 1;
10        while (size < n) size <<= 1;
11        tree.assign(2 * size, 0);
12    }
13
14    void update(int idx, int val) {
15        int p = size + idx;
16        tree[p] = val;
17        for (p >>= 1; p >>= 1)
18            tree[p] = tree[p << 1] + tree[p << 1 | 1];
19    }
20
21    int query(int l, int r) const { // [l, r), 0-indexado
22        int ans = 0;
23        for (l += size, r += size; l < r; l >>= 1, r >>= 1) {
24            if (l & 1) ans += tree[l++];
25            if (r & 1) ans += tree[--r];
26        }
27        return ans;
28    };
29};
```

Ejemplo de uso

Ejemplo: cambiar el valor de un elemento y preguntar la suma de cualquier intervalo.

```
1 SegTree st(5);
2 st.update(0, 10);
3 st.update(1, 20);
4 st.update(2, 30);
5 cout << st.query(0, 3); // 10 + 20 + 30
```

3.4 Segment Tree con Lazy Propagation

Complejidad de Tiempo:

- **Construcción:** $\mathcal{O}(N)$
- **Actualización por Rangos (Range Update):** $\mathcal{O}(\log N)$
- **Consulta por rango (Range Query):** $\mathcal{O}(\log N)$

Complejidad de Espacio: $\mathcal{O}(N)$ (Arreglos de tamaño $4N$). Permite modificaciones masivas con *lazy propagation*. La función `apply` actualiza directamente un nodo y su etiqueta, reduciendo operaciones repetidas de suma y propagación.

```
1 struct SegTreeLazy {
2     int n; vector<ll> tree, lazy;
3     SegTreeLazy(int n) : n(n), tree(4 * n, 0), lazy(4 * n, 0) {}
4
5     inline void apply(int node, int start, int end, ll val) {
6         tree[node] += (end - start + 1LL) * val;
7         lazy[node] += val;
8     }
9
10    inline void push(int node, int start, int end) {
11        if (lazy[node] == 0 || start == end) return;
12        int mid = (start + end) >> 1;
13        ll val = lazy[node];
14        apply(node << 1, start, mid, val);
15        apply(node << 1 | 1, mid + 1, end, val);
16        lazy[node] = 0;
17    }
18
19    void updateRange(int node, int start, int end, int l,
20        int r, ll val) {
21        if (r < start || end < l) return;
22        if (l <= start && end <= r) {
23            apply(node, start, end, val);
24            return;
25        }
26        push(node, start, end);
27        int mid = (start + end) >> 1;
28        updateRange(node << 1, start, mid, l, r, val);
29        updateRange(node << 1 | 1, mid + 1, end, l, r, val);
30        tree[node] = tree[node << 1] + tree[node << 1 | 1];
31    }
32
33    ll queryRange(int node, int start, int end, int l, int r) {
34        if (r < start || end < l) return 0;
35        if (l <= start && end <= r) return tree[node];
36        push(node, start, end);
37        int mid = (start + end) >> 1;
38        return queryRange(node << 1, start, mid, l, r) +
39            queryRange(node << 1 | 1, mid + 1, end, l, r);
40    }
41};
```

Ejemplo de uso

Ejemplo: aumentar todos los salarios de un rango de empleados sin actualizarlos uno por uno.

```
1 SegTreeLazy st(5);
2
3 // Sumar 100 a los empleados 1..3
4 st.updateRange(1, 0, 4, 1, 3, 100);
5
6 cout << st.queryRange(1, 0, 4, 1, 3);
```

3.5 Merge Sort Tree

Complejidad de Tiempo:

- **Construcción:** $\mathcal{O}(N \log N)$
- **Consulta (Query):** $\mathcal{O}(\log^2 N)$ (utilizando búsqueda binaria interna)

Complejidad de Espacio: $\mathcal{O}(N \log N)$ (cada elemento del arreglo pertenece a $\log N$ vectores intermedios).

Estructura donde cada nodo del Segment Tree almacena un vector ordenado con todos los elementos de su subrango. Útil para consultas del tipo "cuántos números menores a K existen en el rango $[L, R]$ ".

```
1 struct MergeSortTree {
2     int n; vector<vector<int>>> tree;
3     MergeSortTree(int n) : n(n), tree(4 * n) {}
4     void build(const vector<int>& arr, int node, int start, int end) {
5         if (start == end) {
6             tree[node] = {arr[start]};
7             return;
8         }
9         int mid = (start + end) / 2;
10        build(arr, 2 * node, start, mid);
11        build(arr, 2 * node + 1, mid + 1, end);
12
13        // Reserva exactamente el tamaño final para evitar
14        // realocaciones.
15        tree[node].resize(end - start + 1);
16        merge(tree[2 * node].begin(), tree[2 * node + 1].end(),
17            tree[2 * node + 1].begin(), tree[2 * node + 1].end(),
18            tree[node].begin());
19    }
20    // Devuelve la cantidad de elementos estrictamente
21    // menores que K en [l, r]
22    int query(int node, int start, int end, int l, int r, int k) {
23        if (r < start || end < l) return 0;
24        if (l <= start && end <= r) {
25            return lower_bound(tree[node].begin(), tree[node].end(), k) - tree[node].begin();
26        }
27        int mid = (start + end) / 2;
28        return query(2 * node, start, mid, l, r, k) +
29            query(2 * node + 1, mid + 1, end, l, r, k);
30    }
31};
```

Ejemplo de uso

Ejemplo: contar cuántos valores menores que K aparecen dentro de un intervalo.

```
1 vector<int> a = {8, 2, 7, 3, 5};
2 MergeSortTree mst(a.size());
3 mst.build(a, 1, 0, a.size() - 1);
4
5 // ¿Cuántos valores menores que 6 hay entre índices 1 y 4?
6 cout << mst.query(1, 0, 4, 1, 4, 6);
```

3.6 Sparse Table (RMQ Estático)

Complejidad de Tiempo:

- **Construcción:** $\mathcal{O}(N \log N)$
- **Consulta (Query):** $\mathcal{O}(1)$

Complejidad de Espacio: $\mathcal{O}(N \log N)$, sin arreglo auxiliar de logs.

Ideal para consultas de rango sobre funciones *idempotentes* (min, max, gcd, AND, OR) cuando el arreglo **no cambia**. Permite responder en $\mathcal{O}(1)$ usando dos ventanas de potencia de dos que se solapan.

```
1 struct SparseTable {
2     vector<vector<int>>> table;
3     int n;
4     SparseTable(const vector<int>& arr) : n((int)arr.size()) {
5         int K = (n == 0 ? 0 : 32 - __builtin_clz(n));
6         table.assign(K, vector<int>(n));
7         if (n == 0) return;
8         table[0] = arr;
9         for (int j = 1; j < K; j++) {
10             int len = 1 << j;
11             for (int i = 0; i + len <= n; i++)
12                 table[j][i] = min(table[j-1][i],
13                                   table[j-1][i + (len >>
14                                     1)]);
15         }
16         // Mínimo en el rango [l, r] (0-indexado, inclusivo).
17         // __builtin_clz evita almacenar un arreglo logs de
18         // tamaño N.
19         int query(int l, int r) const {
20             int len = r - l + 1;
21             int j = 31 - __builtin_clz(len);
22             return min(table[j][l], table[j][r - (1 << j) +
23                       1]);
24         }
25     };
26 }
```

Ejemplo de uso

Ejemplo: consultar rápidamente la temperatura mínima entre dos días cuando el arreglo no cambia.

```
1 vector<int> temperatura = {30, 25, 28, 20, 35};
2 SparseTable st(temperatura);
3
4 cout << st.query(1, 4); // minimo entre dias 1 y 4
```

3.7 DSU con Unión por Tamaño (Rollback-Ready)

Complejidad de Tiempo: $\mathcal{O}(\log N)$ por operación (sin compresión de caminos, para permitir deshacer).

Complejidad de Espacio: $\mathcal{O}(N)$

Variante de DSU que evita la compresión de caminos para poder deshacer uniones (*rollback*), útil en algoritmos offline como Kruskal reconstruible o consultas persistentes de conectividad.

```
1 struct DSURollback {
2     vector<int> parent, sz;
3     vector<pair<int, int>> history; // (nodo modificado,
4     // valor previo del parent)
5     DSURollback(int n) { parent.resize(n); sz.assign(n, 1);
6     // iota(parent.begin(), parent.end(), 0); }
7     int find(int i) { while (parent[i] != i) i = parent[i];
8     return i; }
9     bool unite(int i, int j) {
10         int ri = find(i), rj = find(j);
11         if (ri == rj) { history.push_back({-1, -1});
12         return false; }
13         if (sz[ri] < sz[rj]) swap(ri, rj);
14         history.push_back({rj, parent[rj]});
15         parent[rj] = ri; sz[ri] += sz[rj];
16         return true;
17     }
18     void rollback() {
```

```
15         auto [node, prevParent] = history.back(); history.
16         pop_back();
17         if (node == -1) return;
18         sz[parent[node]] -= sz[node];
19         parent[node] = prevParent;
20     }
21 }
```

Ejemplo de uso

Ejemplo: unir ciudades y después deshacer una unión en un algoritmo offline.

```
1 DSURollback dsu(6);
2
3 dsu.unite(1, 2);
4 dsu.unite(2, 3);
5
6 cout << (dsu.find(1) == dsu.find(3)); // 1
7
8 dsu.rollback(); // deshace la ultima union
```

3.8 Trie (Árbol de Prefijos)

Complejidad de Tiempo: $\mathcal{O}(L)$ por inserción/consulta, donde L es la longitud de la cadena o el número de bits.

Complejidad de Espacio: $\mathcal{O}(N \cdot L)$

Estructura para almacenar conjuntos de cadenas o números binarios y responder consultas de prefijos comunes. La variante mostrada trabaja sobre bits de enteros de 30 posiciones, útil para problemas de XOR máximo.

```
1 struct TrieXor {
2     struct Node { int child[2] = {-1, -1}; };
3     vector<Node> trie;
4     static const int BITS = 30;
5     TrieXor() { trie.push_back(Node()); }
6     void insert(int num) {
7         int cur = 0;
8         for (int i = BITS; i >= 0; i--) {
9             int b = (num >> i) & 1;
10            if (trie[cur].child[b] == -1) {
11                trie[cur].child[b] = trie.size();
12                trie.push_back(Node());
13            }
14            cur = trie[cur].child[b];
15        }
16    }
17    // Devuelve el maximo XOR posible entre 'num' y algun
18    // valor insertado
19    int maxXor(int num) {
20        int cur = 0, res = 0;
21        for (int i = BITS; i >= 0; i--) {
22            int b = (num >> i) & 1;
23            int want = 1 - b;
24            if (trie[cur].child[want] != -1) {
25                res |= (1 << i);
26                cur = trie[cur].child[want];
27            } else {
28                cur = trie[cur].child[b];
29            }
30        }
31        return res;
32    }
33 }
```

Ejemplo de uso

Ejemplo: guardar números y preguntar cuál produce el XOR máximo con un número dado.

```
1 TrieXor trie;
2
3 trie.insert(5);
4 trie.insert(10);
5 trie.insert(25);
```

```
6  
7 cout << trie.maxXor(7);
```

4. Lower Bound y Upper Bound

4.1 Conceptos y Uso Básico en STL

Complejidad de Tiempo: $\mathcal{O}(\log N)$ sobre contenedores de acceso aleatorio (vector, array).

Requisito: el rango debe estar **ordenado** respecto al criterio usado.

`lower_bound(first, last, val)` devuelve un iterador al primer elemento $\geq val$.

`upper_bound(first, last, val)` devuelve un iterador al primer elemento $> val$.

Ambos devuelven `last` si no existe tal elemento. La diferencia entre ambos resultados da la cantidad de ocurrencias exactas de `val`.

```
1 vector<int> v = {1, 3, 3, 3, 5, 7, 9};
2 auto lb = lower_bound(v.begin(), v.end(), 3); // apunta al
  primer 3 (indice 1)
3 auto ub = upper_bound(v.begin(), v.end(), 3); // apunta al
  primer >3 (indice 4, valor 5)
4 int idx_lb = lb - v.begin(); // 1
5 int idx_ub = ub - v.begin(); // 4
6 int cantidad_de_3 = ub - lb; // 3
7 bool existe = (lb != v.end() && *lb == 3); // forma
  estandar de checar existencia
```

Ejemplo de uso

Ejemplo sencillo de aplicación del bloque de código.

```
1 vector<int> v = {1, 3, 3, 3, 5, 7, 9};
2 int x = 3;
3
4 auto lb = lower_bound(v.begin(), v.end(), x);
5 auto ub = upper_bound(v.begin(), v.end(), x);
6
7 cout << "Primer indice >= x: " << (lb - v.begin()) << '\n'
  ;
8 cout << "Cantidad de x: " << (ub - lb) << '\n';
```

Patrones Adaptativos Rápidos

Snippets listos para pegar y adaptar según lo que se necesite consultar:

```
1 // Contar cuantos elementos son <= x
2 int cnt_le(vector<int>& v, int x) { return upper_bound(v.
  begin(), v.end(), x) - v.begin(); }
3 // Contar cuantos elementos son < x
4 int cnt_lt(vector<int>& v, int x) { return lower_bound(v.
  begin(), v.end(), x) - v.begin(); }
5 // Contar cuantos elementos son >= x
6 int cnt_ge(vector<int>& v, int x) { return v.end() -
  lower_bound(v.begin(), v.end(), x); }
7 // Contar cuantos elementos son > x
8 int cnt_gt(vector<int>& v, int x) { return v.end() -
  upper_bound(v.begin(), v.end(), x); }
9 // Contar cuantos elementos hay en el rango [lo, hi]
10 int cnt_range(vector<int>& v, int lo, int hi) {
11   return upper_bound(v.begin(), v.end(), hi) -
     lower_bound(v.begin(), v.end(), lo);
12 }
13 // Mayor elemento <= x (floor). Retorna -1 si no existe
14 int floor_val(vector<int>& v, int x) {
15   auto it = upper_bound(v.begin(), v.end(), x);
16   if (it == v.begin()) return -1;
17   return *prev(it);
18 }
19 // Menor elemento >= x (ceil). Retorna -1 si no existe
20 int ceil_val(vector<int>& v, int x) {
21   auto it = lower_bound(v.begin(), v.end(), x);
22   if (it == v.end()) return -1;
23   return *it;
24 }
```

Ejemplo de uso

Ejemplo directo de uso de `cnt_le` sobre datos pequeños para ver qué resultado produce.

```
1 vector<int> v = {1, 2, 2, 4, 7, 9};
2 cout << cnt_le(v, 4) << '\n'; // 4 elementos <= 4
3 cout << cnt_lt(v, 4) << '\n'; // 3 elementos < 4
4 cout << cnt_range(v, 2, 7) << '\n'; // 5 elementos en
  [2,7]
```

4.2 Búsqueda con Comparador Personalizado

Complejidad de Tiempo: $\mathcal{O}(\log N)$

Cuando el contenedor guarda `pair`, `struct` u otro tipo compuesto, se busca por un campo específico pasando un comparador (lambda) como cuarto argumento. **Importante:** el comparador debe seguir el mismo orden estricto usado al ordenar el contenedor.

```
1 struct Persona { int edad; string nombre; };
2 vector<Persona> gente = { {18,"Ana"}, {20,"Luis"}, {20,"
  Eva"}, {25,"Tom"} };
3 // gente esta ordenado por 'edad'
4 // Buscar primera persona con edad >= 20 (comparando 'val'
  contra el campo del struct)
5 auto it = lower_bound(gente.begin(), gente.end(), 20,
6   [](const Persona& p, int val) { return p.edad < val;
  });
7 // Buscar primera persona con edad > 20
8 auto it2 = upper_bound(gente.begin(), gente.end(), 20,
9   [](int val, const Persona& p) { return val < p.edad;
  });
10 // Sirve igual con vector<pair<int,int>> ordenado por '
  first'
11 vector<pair<int,int>> vp = {{1,5},{3,8},{3,2},{7,1}};
12 auto it3 = lower_bound(vp.begin(), vp.end(), make_pair(3,
  INT_MIN));
```

Ejemplo de uso

Busca un elemento dentro de una estructura ordenada usando el campo que nos interesa.

```
1 auto it = lower_bound(gente.begin(), gente.end(), 20,
2   [](const Persona& p, int x) { return p.edad < x; });
```

4.3 lower_bound / upper_bound en set y multiset

Complejidad de Tiempo: $\mathcal{O}(\log N)$ usando el método miembro (NO usar la función libre `std::lower_bound` sobre estos contenedores: al no tener iteradores de acceso aleatorio, degrada a $\mathcal{O}(N)$).

`set/multiset` ya están ordenados internamente; siempre se debe llamar al **método miembro** `.lower_bound()` / `.upper_bound()` para mantener eficiencia.

```
1 multiset<int> ms = {1, 3, 3, 5, 7};
2 auto it = ms.lower_bound(3); // 0(log N), correcto
3 // auto it_malo = lower_bound(ms.begin(), ms.end(), 3); //
  O(N), EVITAR
4 // Predecesor y sucesor de x (aunque x no este en el set)
5 int predecesor(multiset<int>& s, int x) {
6   auto it = s.lower_bound(x);
7   if (it == s.begin()) return INT_MIN; // no hay
  predecesor
```

```

8     return *prev(it);
9 }
10 int sucesor(multiset<int>& s, int x) {
11     auto it = s.upper_bound(x);
12     if (it == s.end()) return INT_MAX; // no hay sucesor
13     return *it;
14 }
15 // Eliminar SOLO una ocurrencia de x en un multiset (erase
16 // (x) borra TODAS)
17 auto itDel = ms.find(3);
18 if (itDel != ms.end()) ms.erase(itDel);

```

Ejemplo de uso

Consulta directamente el siguiente o anterior elemento en un conjunto ordenado.

```

1 multiset<int> s = {1, 3, 3, 5, 7};
2 auto it = s.lower_bound(4);
3 cout << *it;

```

4.4 Búsqueda Binaria Genérica sobre Predicado (Adaptativa)

Complejidad de Tiempo: $\mathcal{O}(\log N \cdot f)$, donde f es el costo de evaluar el predicado (*check*).

Complejidad de Espacio: $\mathcal{O}(1)$

Esta plantilla es la más **reutilizable** en competencia: sirve para cualquier situación monótona (no solo arreglos ordenados), como *binary search on answer*. Solo se debe definir la función *check(x)* que sea *true* para todo $x \geq$ respuesta y *false* antes de ella (patrón FFFF...TTTT).

```

1 // Encuentra el menor x en [lo, hi] tal que check(x) ==
2 // true
3 // (asume que check es monotonamente: false,false,...,false,
4 // true,true,...,true)
5 template <typename T, typename F>
6 T binary_search_first_true(T lo, T hi, F check) {
7     T ans = hi + 1; // valor centinela si nunca se cumple
8     while (lo <= hi) {
9         T mid = lo + (hi - lo) / 2;
10        if (check(mid)) { ans = mid; hi = mid - 1; }
11        else lo = mid + 1;
12    }
13    return ans;
14 }
15 // Ejemplo de uso: menor capacidad de barco para repartir
16 // paquetes en D días
17 // bool check(int capacidad) { ... simula si alcanza con
18 // esa capacidad ... }
19 // int resp = binary_search_first_true(minCap, maxCap,
20 // check);
21 // Variante para punto flotante (numero fijo de
22 // iteraciones en vez de lo<=hi)
23 template <typename F>
24 double binary_search_real(double lo, double hi, F check,
25 int iters = 100) {
26     for (int i = 0; i < iters; i++) {
27         double mid = (lo + hi) / 2;
28         if (check(mid)) hi = mid; else lo = mid;
29     }
30     return lo;
31 }

```

Ejemplo de uso

Ejemplo: encontrar la capacidad mínima de un camión que permite transportar todos los paquetes.

```

1 auto check = [&](long long capacidad) {
2     // Simular el reparto y devolver true si alcanza.
3     return capacidad >= 100;
4 };

```

```

5 long long respuesta =
6     binary_search_first_true(1LL, 1000LL, check);
7

```

4.5 Implementación Manual (Referencia Interna)

Complejidad de Tiempo: $\mathcal{O}(\log N)$

Útil para entender qué hace la STL por dentro, o para adaptarla a estructuras sin iteradores estándar (ej. arreglos con índices *custom*, matrices implícitas, respuestas indexadas por función).

```

1 // Version manual de lower_bound: primer indice con arr[i]
2 // >= target
3 int my_lower_bound(const vector<int>& arr, int target) {
4     int lo = 0, hi = (int)arr.size(); // hi = size (no
5     // size-1): rango [lo, hi)
6     while (lo < hi) {
7         int mid = lo + (hi - lo) / 2;
8         if (arr[mid] < target) lo = mid + 1;
9         else hi = mid;
10    }
11    return lo; // si lo == arr.size(), no existe tal
12    // elemento
13 }
14 // Version manual de upper_bound: primer indice con arr[i]
15 // > target
16 int my_upper_bound(const vector<int>& arr, int target) {
17     int lo = 0, hi = (int)arr.size();
18     while (lo < hi) {
19         int mid = lo + (hi - lo) / 2;
20         if (arr[mid] <= target) lo = mid + 1;
21         else hi = mid;
22    }
23    return lo;
24 }

```

Ejemplo de uso

Permite entender o reutilizar la búsqueda binaria sin depender de la STL.

```

1 vector<int> a = {1, 3, 5, 8};
2 int pos = my_lower_bound(a, 5);
3 cout << pos;

```

5. Algoritmos de Grafos

5.1 Tabla de Decisión Rápida

Antes de programar, ubica el problema en esta tabla. Ahorra minutos valiosos en el momento del concurso.

Pista en el enunciado	Algoritmo
Menor cantidad de pasos/aristas, sin pesos	BFS
Pesos todos iguales a 1 o dos valores fijos	BFS 0/1 (deque)
Varios orígenes simultáneos	BFS multi-fuente
Camino mínimo con pesos ≥ 0	Dijkstra
Camino mínimo con pesos negativos	Bellman-Ford
Todos los pares de distancias, $V \leq 500$	Floyd-Warshall
¿Se puede llegar de A a B?	DFS o BFS
Contar componentes conexas / islas en grid	DFS o BFS desde cada no visitado
¿Es bipartito? / 2 colores	BFS/DFS coloreando
Orden de tareas con dependencias	Topological Sort (Kahn o DFS)
Detectar ciclo	DFS (colores) o Kahn
Árbol de conexión de costo mínimo	Kruskal o Prim
Componentes fuertemente conexas (dirigido)	Tarjan / Kosaraju
Nodo/arista crítica que desconecta el grafo	Puentes y Puntos de Articulación
Ancastro común de dos nodos en árbol	LCA (Binary Lifting)
Satisfacibilidad de restricciones OR con 2 opciones	2-SAT
Flujo máximo / corte mínimo	Edmonds-Karp
Grid/matriz con movimientos (arriba/abajo/izq/der)	Grafo implícito: BFS/DFS sobre celdas

5.2 Plantilla Universal de Grafo

Copiar y pegar según lo que se necesite. Usar SIEMPRE la misma variable adj en toda la solución para no confundirse a mitad de la competencia.

```
1 int n, m; // nodos y aristas
2 vector<vector<int>>> adj; // grafo NO ponderado
3 vector<vector<pair<int,int>>> adjW; // grafo ponderado: {vecino, peso}
4
5 void leerGrafoNoDirigido(bool ponderado) {
6     cin >> n >> m;
7     adj.assign(n + 1, {});
8     adjW.assign(n + 1, {});
9     for (int i = 0; i < m; i++) {
10         int u, v; cin >> u >> v;
11         if (ponderado) {
12             int w; cin >> w;
13             adjW[u].push_back({v, w});
14             adjW[v].push_back({u, w});
15         } else {
16             adj[u].push_back(v);
17             adj[v].push_back(u); // quitar esta linea si es dirigido
18         }
19     }
20 }
21 // IMPORTANTE (multi-test): siempre limpiar/reasignar adj y adjW entre casos de prueba.
```

Ejemplo de uso

Ejemplo directo de uso de leerGrafoNoDirigido sobre datos pequeños para ver qué resultado produce.

```
1 // Entrada de prueba:
2 // 4 3
3 // 1 2
4 // 2 3
5 // 2 4
6
7 istream in("4 3\n1 2\n2 3\n2 4\n");
8 cin.rdbuf(in.rdbuf());
9 leerGrafoNoDirigido(false);
10
11 for (int u = 1; u <= n; ++u) {
12     cout << u << ": ";
13     for (int v : adj[u]) cout << v << ' ';
14     cout << '\n';
15 }
```

5.3 BFS (Búsqueda en Anchura)

Complejidad: $\mathcal{O}(V + E)$ tiempo, $\mathcal{O}(V)$ espacio.

Garantiza la distancia mínima en aristas cuando todos los pesos son 1 (o no hay pesos).

```
1 vector<int> dist_, parent_;
2 void bfs(int start, int n, vector<vector<int>>& adj) {
3     dist_.assign(n + 1, -1);
4     parent_.assign(n + 1, -1);
5     queue<int> q;
6     dist_[start] = 0;
7     q.push(start);
8     while (!q.empty()) {
9         int u = q.front(); q.pop();
10        for (int v : adj[u]) {
11            if (dist_[v] == -1) {
12                dist_[v] = dist_[u] + 1;
13                parent_[v] = u;
14                q.push(v);
15            }
16        }
17    }
18 }
19 // Reconstruir camino start -> target (vacío si no hay camino)
20 vector<int> reconstruirCamino(int target) {
21     if (dist_[target] == -1) return {};
22     vector<int> camino;
23     for (int v = target; v != -1; v = parent_[v]) camino.push_back(v);
24     reverse(camino.begin(), camino.end());
25     return camino;
26 }
```

Ejemplo de uso

Ejemplo: construimos un grafo pequeño usando pares nodo, vecino, ejecutamos BFS desde el nodo 0 y al final reconstruimos e imprimimos el recorrido hasta el nodo 5.

```
1 pair<int,int> par1 = {0, 1};
2 pair<int,int> par2 = {0, 2};
3 pair<int,int> par3 = {1, 3};
4 pair<int,int> par4 = {2, 4};
5 pair<int,int> par5 = {3, 5};
6 pair<int,int> par6 = {4, 5};
7
8 vector<vector<int>>> adj(6);
9 adj[par1.first].push_back(par1.second);
10 adj[par2.first].push_back(par2.second);
11 adj[par3.first].push_back(par3.second);
12 adj[par4.first].push_back(par4.second);
13 adj[par5.first].push_back(par5.second);
14 adj[par6.first].push_back(par6.second);
15
```

```

16 // Si el grafo es no dirigido, agregar tambien la vuelta.
17 adj[1].push_back(0);
18 adj[2].push_back(0);
19 adj[3].push_back(1);
20 adj[4].push_back(2);
21 adj[5].push_back(3);
22 adj[5].push_back(4);
23
24 bfs(0, 6, adj);
25
26 vector<int> camino = reconstruirCamino(5);
27 for (int v : camino)
28     cout << v << ' ';
29
30 // Salida: 0 1 3 5

```

5.3.1 BFS Multi-Fuente

Útil cuando hay varios orígenes que "expanden" simultáneamente (ej. varias fuentes de fuego, virus, etc.). Basta con meter todos los orígenes a la cola desde el inicio con distancia 0.

```

1 vector<int> bfsMultiFuente(int n, vector<vector<int>>& adj
  , vector<int>& fuentes) {
2     vector<int> dist(n + 1, -1);
3     queue<int> q;
4     for (int s : fuentes) { dist[s] = 0; q.push(s); }
5     while (!q.empty()) {
6         int u = q.front(); q.pop();
7         for (int v : adj[u]) {
8             if (dist[v] == -1) { dist[v] = dist[u] + 1; q.
          push(v); }
9         }
10    }
11    return dist;
12 }

```

Ejemplo de uso

Ejemplo: varias estaciones de bomberos atienden simultáneamente a las ciudades más cercanas.

```

1 vector<int> estaciones = {1, 5, 8};
2 vector<int> dist = bfsMultiFuente(n, adj, estaciones);
3
4 cout << dist[10];           // distancia a la estacion
    mas cercana

```

5.3.2 BFS 0/1 (Deque)

Complejidad: $\mathcal{O}(V + E)$

Cuando las aristas solo tienen peso 0 o 1 (o dos valores fijos), es más rápido que Dijkstra y evita la cola de prioridad. Aristas de peso 0 se insertan al frente del deque, las de peso 1 al final.

```

1 vector<int> bfs01(int start, int n, vector<vector<pair<int
  , int>>>& adjW) {
2     vector<int> dist(n + 1, INT_MAX);
3     deque<int> dq;
4     dist[start] = 0;
5     dq.push_back(start);
6     while (!dq.empty()) {
7         int u = dq.front(); dq.pop_front();
8         for (auto [v, w] : adjW[u]) { // w debe ser 0 o 1
9             if (dist[u] + w < dist[v]) {
10                dist[v] = dist[u] + w;
11                if (w == 0) dq.push_front(v);
12                else dq.push_back(v);
13            }
14        }
15    }
16    return dist;
17 }

```

Ejemplo de uso

Ejemplo: moverse por un mapa donde algunas transiciones cuestan 0 y otras cuestan 1.

```

1 vector<vector<pair<int,int>>> adjW(n + 1);
2 vector<int> dist = bfs01(1, n, adjW);
3
4 cout << dist[n];

```

5.4 DFS (Búsqueda en Profundidad)

Complejidad: $\mathcal{O}(V + E)$ tiempo, $\mathcal{O}(V)$ espacio.

Cuidado: con $n \gtrsim 10^5$ y grafos tipo "cadena", la version recursiva puede tronar por stack overflow. Usar la version iterativa en ese caso.

```

1 vector<bool> visitado;
2 void dfs(int u, vector<vector<int>>& adj) {
3     visitado[u] = true;
4     // procesar u aqui
5     for (int v : adj[u]) {
6         if (!visitado[v]) dfs(v, adj);
7     }
8 }

```

Ejemplo de uso

Ejemplo: recorrer una red y visitar todos los nodos alcanzables desde una ciudad.

```

1 visitado.assign(n + 1, false);
2 dfs(1, adj);
3
4 if (visitado[destino])
5     cout << "Existe camino";

```

5.4.1 DFS Iterativo (evita stack overflow)

```

1 void dfsIterativo(int start, int n, vector<vector<int>>&
  adj) {
2     vector<bool> visitado(n + 1, false);
3     stack<int> st;
4     st.push(start);
5     while (!st.empty()) {
6         int u = st.top(); st.pop();
7         if (visitado[u]) continue;
8         visitado[u] = true;
9         // procesar u aqui
10        for (int v : adj[u]) {
11            if (!visitado[v]) st.push(v);
12        }
13    }
14 }

```

Ejemplo de uso

Úsalo cuando el grafo puede tener cientos de miles de nodos y una DFS recursiva podría agotar la pila.

```

1 dfsIterativo(1, n, adj);
2 // Todos los nodos alcanzables desde 1 quedaron marcados.

```

5.5 Componentes Conexas y Grafos Bipartitos

Complejidad: $\mathcal{O}(V + E)$

```

1 // Cuenta componentes conexas y asigna a cada nodo su id
  de componente
2 int contarComponentes(int n, vector<vector<int>>& adj,
  vector<int>& compId) {
3     compId.assign(n + 1, -1);
4     int comp = 0;
5     for (int i = 1; i <= n; i++) {
6         if (compId[i] != -1) continue;
7         queue<int> q; q.push(i); compId[i] = comp;
8         while (!q.empty()) {
9             int u = q.front(); q.pop();
10            for (int v : adj[u]) {
11                if (compId[v] == -1) { compId[v] = comp; q
                .push(v); }
12            }
13        }
14        comp++;
15    }
16    return comp;
17 }
18 // Determina si el grafo es bipartito (coloreable con 2
  colores)
19 bool esBipartito(int n, vector<vector<int>>& adj) {
20     vector<int> color(n + 1, -1);
21     for (int i = 1; i <= n; i++) {
22         if (color[i] != -1) continue;
23         queue<int> q; q.push(i); color[i] = 0;
24         while (!q.empty()) {
25             int u = q.front(); q.pop();
26             for (int v : adj[u]) {
27                 if (color[v] == -1) { color[v] = 1 - color
                [u]; q.push(v); }
28                 else if (color[v] == color[u]) return
                false;
29             }
30         }
31     }
32     return true;
33 }
```

Ejemplo de uso

Ejemplo: contar grupos desconectados y comprobar si se pueden dividir en dos equipos.

```

1 vector<int> compId;
2 int grupos = contarComponentes(n, adj, compId);
3
4 bool dosEquipos = esBipartito(n, adj);
```

5.6 Detección de Ciclos

Complejidad: $\mathcal{O}(V + E)$

5.6.1 Grafo NO Dirigido

Se compara con el padre para no contar la misma arista dos veces.

```

1 vector<bool> visitado;
2 bool cicloNoDirigido(int u, int padre, vector<vector<int>
  >>& adj) {
3     visitado[u] = true;
4     for (int v : adj[u]) {
5         if (!visitado[v]) {
6             if (cicloNoDirigido(v, u, adj)) return true;
7         } else if (v != padre) {
8             return true;
9         }
10    }
11    return false;
12 }
```

```

13 bool tieneCicloNoDirigido(int n, vector<vector<int>>& adj)
14 {
15     visitado.assign(n + 1, false);
16     for (int i = 1; i <= n; i++)
17         if (!visitado[i] && cicloNoDirigido(i, -1, adj))
18             return true;
19     return false;
20 }
```

Ejemplo de uso

Ejemplo: detectar si una red de carreteras contiene algún ciclo.

```

1 bool hayCiclo = tieneCicloNoDirigido(n, adj);
2
3 cout << (hayCiclo ? "SI" : "NO");
```

5.6.2 Grafo Dirigido (3 colores)

Un ciclo existe solo si se llega a un nodo que sigue "en la pila de recursión" (gris), no uno ya terminado (negro).

```

1 // 0 = blanco, 1 = gris (en el camino actual), 2 = negro (
  terminado)
2 vector<int> color_;
3 bool cicloDirigido(int u, vector<vector<int>>& adj) {
4     color_[u] = 1;
5     for (int v : adj[u]) {
6         if (color_[v] == 1) return true;
7         if (color_[v] == 0 && cicloDirigido(v, adj))
8             return true;
9     }
10    color_[u] = 2;
11    return false;
12 }
13 bool tieneCicloDirigido(int n, vector<vector<int>>& adj) {
14     color_.assign(n + 1, 0);
15     for (int i = 1; i <= n; i++)
16         if (color_[i] == 0 && cicloDirigido(i, adj))
17             return true;
18     return false;
19 }
```

Ejemplo de uso

Ejemplo: comprobar si las dependencias entre tareas forman un ciclo.

```

1 bool hayCiclo = tieneCicloDirigido(n, adj);
2
3 cout << (hayCiclo ? "Dependencias invalidas" : "Orden
  posible");
```

5.7 Ordenamiento Topológico

Solo aplica a DAGs (grafos dirigidos acíclicos).

5.7.1 Kahn (BFS) — Recomendado

Complejidad: $\mathcal{O}(V + E)$

Ventaja: detecta ciclos automáticamente (si orden.size() != n, no es DAG).

```

1 vector<int> topoSortKahn(int n, vector<vector<int>>& adj)
2 {
3     vector<int> indeg(n + 1, 0);
4     for (int u = 1; u <= n; u++) for (int v : adj[u])
5         indeg[v]++;
6     queue<int> q;
7     for (int i = 1; i <= n; i++) if (indeg[i] == 0) q.push
8         (i);
9     vector<int> orden;
10    while (!q.empty()) {
11        int u = q.front(); q.pop();
12        orden.push_back(u);
13        for (int v : adj[u]) indeg[v]--;
14        if (indeg[v] == 0) q.push(v);
15    }
16    return orden;
17 }
```

```

7   while (!q.empty()) {
8       int u = q.front(); q.pop();
9       orden.push_back(u);
10      for (int v : adj[u]) if (--indeg[v] == 0) q.push(v);
11  }
12  if ((int)orden.size() != n) return {}; // hay ciclo,
13  // no es DAG
14  return orden;
15 }

```

Ejemplo de uso

Ejemplo: obtener un orden válido para cursar materias con prerrequisitos.

```

1 vector<int> orden = topoSortKahn(n, adj);
2
3 if (orden.empty())
4     cout << "Hay un ciclo";
5 else
6     for (int materia : orden) cout << materia << ' ';

```

5.7.2 DFS (post-orden invertido)

```

1 vector<bool> visitadoTopo;
2 vector<int> ordenTopo;
3 void dfsTopo(int u, vector<vector<int>>& adj) {
4     visitadoTopo[u] = true;
5     for (int v : adj[u]) if (!visitadoTopo[v]) dfsTopo(v, adj);
6     ordenTopo.push_back(u);
7 }
8 vector<int> topoSortDFS(int n, vector<vector<int>>& adj) {
9     visitadoTopo.assign(n + 1, false);
10    ordenTopo.clear();
11    for (int i = 1; i <= n; i++) if (!visitadoTopo[i]) dfsTopo(i, adj);
12    reverse(ordenTopo.begin(), ordenTopo.end());
13    return ordenTopo; // NO valida ciclos, verificar con
14    // tieneCicloDirigido primero

```

Ejemplo de uso

Ejemplo: construir un orden topológico cuando ya sabemos que el grafo es acíclico.

```

1 vector<int> orden = topoSortDFS(n, adj);
2
3 for (int x : orden) cout << x << ' ';

```

5.8 Caminos Mínimos con Pesos

5.8.1 Dijkstra (pesos no negativos)

Complejidad: $\mathcal{O}((V + E) \log V)$

```

1 const ll INF_LL = 1e18;
2 vector<ll> dijkstra(int src, int n,
3     const vector<vector<pair<int, ll>>>&
4     adj) {
5     vector<ll> dist(n + 1, INF_LL);
6     priority_queue<pair<ll, int>, vector<pair<ll, int>>,
7     greater<>> pq;
8     dist[src] = 0;
9     pq.push({0, src});
10
11    while (!pq.empty()) {
12        auto [d, u] = pq.top();
13        pq.pop();
14        if (d != dist[u]) continue; // descarta entradas
15        // obsoletas
16
17        for (const auto& [v, w] : adj[u]) {

```

```

15         if (d + w < dist[v]) {
16             dist[v] = d + w;
17             pq.push({dist[v], v});
18         }
19     }
20 }
21 return dist;
22 }

```

Ejemplo de uso

Ejemplo: encontrar la ruta más barata entre ciudades cuando las carreteras tienen costos no negativos.

```

1 vector<ll> dist = dijkstra(1, n, adjW);
2
3 cout << dist[n]; // costo mínimo desde 1
4     hasta n

```

5.8.2 Bellman-Ford (permite pesos negativos)

Complejidad: $\mathcal{O}(V \cdot E)$ en el peor caso; puede terminar antes si no hay más relajaciones.

```

1 struct Edge { int u, v; ll w; };
2 pair<vector<ll>, bool> bellmanFord(int src, int n, vector<
3     Edge>& edges) {
4     vector<ll> dist(n + 1, INF_LL);
5     dist[src] = 0;
6     for (int i = 0; i < n - 1; i++) {
7         bool changed = false;
8         for (const auto& e : edges) {
9             if (dist[e.u] != INF_LL && dist[e.u] + e.w <
10             dist[e.v]) {
11                 dist[e.v] = dist[e.u] + e.w;
12                 changed = true;
13             }
14         }
15         if (!changed) break; // ya convergió; evita
16         // pasadas innecesarias
17     }
18     bool cicloNegativo = false;
19     for (const auto& e : edges)
20         if (dist[e.u] != INF_LL && dist[e.u] + e.w < dist[
21         e.v]) { cicloNegativo = true; break; }
22     return {dist, cicloNegativo};
23 }

```

Ejemplo de uso

Ejemplo: calcular rutas cuando existen descuentos que se representan con pesos negativos.

```

1 auto [dist, cicloNegativo] =
2     bellmanFord(1, n, edges);
3
4 if (cicloNegativo) cout << "Ciclo negativo";
5 else cout << dist[n];

```

5.8.3 Floyd-Warshall (todos los pares)

Complejidad: $\mathcal{O}(V^3)$. Práctico para $V \lesssim 500$.

```

1 void floydWarshall(vector<vector<ll>>& dist, int n) {
2     // dist[i][j] = peso i->j, INF si no hay arista, 0 si
3     // i == j
4     for (int k = 0; k < n; k++) {
5         for (int i = 0; i < n; i++) {
6             if (dist[i][k] == INF_LL) continue;
7             const ll dik = dist[i][k];
8             for (int j = 0; j < n; j++) {
9                 if (dist[k][j] == INF_LL) continue;
10                dist[i][j] = min(dist[i][j], dik + dist[k]
11                [j]);
12            }
13        }
14    }

```

```

11     }
12 }
13 // Ciclo negativo si dist[i][i] < 0 para algun i
14 }

```

Ejemplo de uso

Ejemplo: obtener la distancia mínima entre cualquier par de ciudades.

```

1 floydWarshall(dist, n);
2
3 cout << dist[2][5]; // mejor ruta 2 -> 5

```

5.9 Árbol de Expansión Mínima (MST)

5.9.1 Kruskal (recomendado, más fácil de codear)

Complejidad: $\mathcal{O}(E \log E)$. Requiere DSU (ver capítulo de Estructuras de Datos).

```

1 struct EdgeW { int u, v; ll w; bool operator<(const EdgeW&
2   o) const { return w < o.w; } };
3 ll kruskal(int n, vector<EdgeW>& edges) {
4     sort(edges.begin(), edges.end());
5     DSU dsu(n + 1);
6     ll costoTotal = 0; int aristasUsadas = 0;
7     for (const auto& e : edges) {
8         if (dsu.find(e.u) != dsu.find(e.v)) {
9             dsu.unite(e.u, e.v);
10            costoTotal += e.w;
11            if (++aristasUsadas == n - 1) break; // MST
12            completa
13        }
14        // Si aristasUsadas != n - 1, el grafo no es conexo
15        return costoTotal;
16    }
17 }

```

Ejemplo de uso

Ejemplo: conectar todas las ciudades gastando el mínimo posible en carreteras.

```

1 sort(edges.begin(), edges.end());
2 ll costo = kruskal(n, edges);
3
4 cout << costo;

```

5.9.2 Prim (mejor para grafos densos)

Complejidad: $\mathcal{O}(E \log V)$

```

1 ll prim(int n, vector<vector<pair<int,ll>>>& adj) {
2     vector<bool> inMST(n + 1, false);
3     priority_queue<pair<ll,int>, vector<pair<ll,int>>,
4       greater<>> pq;
5     pq.push({0, 1}); // asumiendo raiz = nodo 1
6     ll costoTotal = 0;
7     while (!pq.empty()) {
8         auto [w, u] = pq.top(); pq.pop();
9         if (inMST[u]) continue;
10        inMST[u] = true; costoTotal += w;
11        for (auto [v, peso] : adj[u]) if (!inMST[v]) pq.
12          push({peso, v});
13    }
14    return costoTotal;
15 }

```

Ejemplo de uso

Ejemplo: construir una red mínima de cables conectando todos los edificios.

```

1 ll costo = prim(n, adjW);
2 cout << costo;

```

5.10 Componentes Fuertemente Conexas: Tarjan

Complejidad: $\mathcal{O}(V + E)$. Encuentra SCC de un grafo dirigido en una sola pasada DFS.

```

1 int timerScc; vector<vector<int>> adjScc;
2 vector<int> disc, low, comp;
3 vector<bool> onStack; stack<int> stScc; int sccCount;
4 void tarjanDfs(int u) {
5     disc[u] = low[u] = timerScc++;
6     stScc.push(u); onStack[u] = true;
7     for (int v : adjScc[u]) {
8         if (disc[v] == -1) { tarjanDfs(v); low[u] = min(
9           low[u], low[v]); }
10        else if (onStack[v]) low[u] = min(low[u], disc[v])
11    }
12    if (low[u] == disc[u]) {
13        while (true) {
14            int v = stScc.top(); stScc.pop(); onStack[v] =
15              false;
16            comp[v] = sccCount;
17            if (v == u) break;
18        }
19        sccCount++;
20    }
21 }
22 void tarjanSCC(int n) {
23     disc.assign(n + 1, -1); low.assign(n + 1, -1);
24     comp.assign(n + 1, -1); onStack.assign(n + 1, false);
25     timerScc = 0; sccCount = 0;
26     for (int i = 1; i <= n; i++) if (disc[i] == -1)
27       tarjanDfs(i);
28 }

```

Ejemplo de uso

Ejemplo: encontrar grupos de páginas web donde cada página puede llegar a las demás.

```

1 tarjanSCC(n);
2
3 cout << "La pagina 3 pertenece a SCC "
4     << comp[3];

```

5.11 Puentes y Puntos de Articulación

Complejidad: $\mathcal{O}(V + E)$. Aristas/vértices cuya eliminación desconecta el grafo (grafo NO dirigido).

```

1 int timerBridge; vector<vector<int>> adjBridge;
2 vector<int> discB, lowB, parentB;
3 vector<bool> esArticulacion;
4 vector<pair<int,int>> puentes;
5 void bridgeDfs(int u) {
6     discB[u] = lowB[u] = timerBridge++;
7     int hijos = 0;
8     for (int v : adjBridge[u]) {
9         if (v == parentB[u]) continue;
10        if (discB[v] != -1) {
11            lowB[u] = min(lowB[u], discB[v]);
12        } else {
13            hijos++;
14            parentB[v] = u;
15        }
16    }
17 }

```

```

15     bridgeDfs(v);
16     lowB[u] = min(lowB[u], lowB[v]);
17     if (lowB[v] > discB[u]) puentes.push_back({u,
18         v});
19     if (parentB[u] != -1 && lowB[v] >= discB[u])
20     esArticulacion[u] = true;
21     }
22     if (parentB[u] == -1 && hijos > 1) esArticulacion[u] =
23     true; // caso raiz
24 void encontrarPuentesYArticulacion(int n) {
25     discB.assign(n + 1, -1); lowB.assign(n + 1, -1);
26     parentB.assign(n + 1, -1); esArticulacion.assign(n +
27     1, false);
28     timerBridge = 0;
29     for (int i = 1; i <= n; i++) if (discB[i] == -1)
30     bridgeDfs(i);
31 }

```

Ejemplo de uso

Ejemplo: detectar carreteras críticas cuya eliminación desconecta una red.

```

1 encontrarPuentesYArticulacion(n);
2
3 for (auto [u, v] : puentes)
4     cout << u << " - " << v << '\n';

```

5.12 Ancestro Común Más Bajo (LCA) — Binary Lifting

Complejidad: $\mathcal{O}(V \log V)$ preprocesamiento, $\mathcal{O}(\log V)$ por consulta.

```

1 int LOG_;
2 vector<vector<int>> upLca, adjLca;
3 vector<int> depthLca;
4 void lcaDfs(int u, int p) {
5     upLca[u][0] = p;
6     for (int j = 1; j < LOG_; j++)
7         upLca[u][j] = (upLca[u][j-1] == -1) ? -1 : upLca[
8         upLca[u][j-1]][j-1];
9     for (int v : adjLca[u]) if (v != p) { depthLca[v] =
10     depthLca[u] + 1; lcaDfs(v, u); }
11 }
12 void lcaInit(int n, int raiz) {
13     LOG_ = ceil(log2(max(2, n))) + 1;
14     upLca.assign(n + 1, vector<int>(LOG_, -1));
15     depthLca.assign(n + 1, 0);
16     lcaDfs(raiz, -1);
17 }
18 int lcaQuery(int u, int v) {
19     if (depthLca[u] < depthLca[v]) swap(u, v);
20     int diff = depthLca[u] - depthLca[v];
21     for (int j = 0; j < LOG_; j++) if ((diff >> j) & 1) u
22     = upLca[u][j];
23     if (u == v) return u;
24     for (int j = LOG_ - 1; j >= 0; j--)
25         if (upLca[u][j] != upLca[v][j]) { u = upLca[u][j];
26         v = upLca[v][j]; }
27     return upLca[u][0];
28 }
29 // Distancia entre u y v en el arbol:
30 // int distArbol(int u, int v) { int l = lcaQuery(u, v);
31 // return depthLca[u] + depthLca[v] - 2 * depthLca[l]; }

```

Ejemplo de uso

Ejemplo: encontrar el ancestro común de dos empleados en un árbol jerárquico.

```

1 lcaInit(n, 1);
2
3 int jefeComun = lcaQuery(12, 19);
4 cout << jefeComun;

```

5.13 2-SAT

Complejidad: $\mathcal{O}(V + E)$

```

1 struct TwoSAT {
2     int n;
3     vector<vector<int>> adj, radj;
4     vector<int> comp, order_, assign_;
5     vector<bool> visited;
6     TwoSAT(int n) : n(n), adj(2*n), radj(2*n), comp(2*n,
7     -1), visited(2*n, false) {}
8     int idx(int var, bool val) { return 2*var + (val ? 1 :
9     0); }
10    void addClause(int var1, bool val1, int var2, bool
11    val2) {
12        adj[idx(var1, !val1)].push_back(idx(var2, val2));
13        adj[idx(var2, !val2)].push_back(idx(var1, val1));
14        radj[idx(var2, val2)].push_back(idx(var1, !val1));
15        radj[idx(var1, val1)].push_back(idx(var2, !val2));
16    }
17    void dfs1(int u) {
18        visited[u] = true;
19        for (int v : adj[u]) if (!visited[v]) dfs1(v);
20        order_.push_back(u);
21    }
22    void dfs2(int u, int c) {
23        comp[u] = c;
24        for (int v : radj[u]) if (comp[v] == -1) dfs2(v, c
25        );
26    }
27    bool solve() {
28        for (int i = 0; i < 2*n; i++) if (!visited[i])
29        dfs1(i);
30        int c = 0;
31        for (int i = 2*n - 1; i >= 0; i--) {
32            int u = order_[i];
33            if (comp[u] == -1) dfs2(u, c++);
34        }
35        assign_.assign(n, -1);
36        for (int i = 0; i < n; i++) {
37            if (comp[idx(i, true)] == comp[idx(i, false)])
38            return false;
39            assign_[i] = comp[idx(i, true)] > comp[idx(i,
40            false)];
41        }
42        return true;
43    }
44 };

```

Ejemplo de uso

Ejemplo: elegir valores booleanos que satisfagan restricciones como (A o B).

```

1 TwoSAT sat(3);
2
3 sat.addClause(0, true, 1, false);
4 sat.addClause(1, true, 2, true);
5
6 if (sat.solve())
7     cout << "Existe una asignacion";
8 else
9     cout << "No existe";

```

5.14 Flujo Máximo: Edmonds-Karp

Complejidad: $\mathcal{O}(VE^2)$

```

1 const int INF = 1e9;
2 int nFlujo; vector<vector<int>> capacidad, adjFlujo;
3 int bfsFlujo(int s, int t, vector<int>& parent) {
4     fill(parent.begin(), parent.end(), -1); parent[s] =
5     -2;
6     queue<pair<int, int>> q; q.push({s, INF});
7     while (!q.empty()) {
8         auto [cur, flow] = q.front(); q.pop();
9         for (int next : adjFlujo[cur]) {
10            if (parent[next] == -1 && capacidad[cur][next]
11            > 0) {
12                parent[next] = cur;

```

```

11         int newFlow = min(flow, capacidad[cur][
12             next]);
13         if (next == t) return newFlow;
14         q.push({next, newFlow});
15     }
16 }
17 return 0;
18 }
19 int edmondsKarp(int s, int t) {
20     int flow = 0; vector<int> parent(nFlujo); int newFlow;
21     while ((newFlow = bfsFlujo(s, t, parent))) {
22         flow += newFlow; int cur = t;
23         while (cur != s) {
24             int prev = parent[cur];
25             capacidad[prev][cur] -= newFlow;
26             capacidad[cur][prev] += newFlow;
27             cur = prev;
28         }
29     }
30     return flow;
31 }

```

Ejemplo de uso

Ejemplo: calcular cuántas unidades de agua pueden enviarse desde una fuente hasta un destino.

```

1 int maxFlow = edmondsKarp(origen, destino);
2 cout << maxFlow;

```

5.15 Grafos Implícitos en Grid/Matriz

Complejidad: $\mathcal{O}(R \cdot C)$ donde R, C son filas y columnas. Muy frecuente en competencia: laberintos, conteo de islas, agua/incendio expandiéndose. Cada celda es un nodo implícito, sin necesidad de construir adj explícitamente.

```

1 int R, C;
2 vector<string> grid; // grid[i][j] = caracter de la celda
3 vector<vector<int>> distGrid;
4 // 4 direcciones (arriba, abajo, izquierda, derecha)
5 int dx4[] = {-1, 1, 0, 0};
6 int dy4[] = {0, 0, -1, 1};
7 // 8 direcciones (agrega diagonales)
8 int dx8[] = {-1,-1,-1, 0, 0, 1, 1, 1};
9 int dy8[] = {-1, 0, 1,-1, 1,-1, 0, 1};
10
11 bool dentroDelGrid(int x, int y) { return x >= 0 && x < R
    && y >= 0 && y < C; }
12
13 void bfsGrid(int startX, int startY, char obstaculo = '#')
14 {
15     distGrid.assign(R, vector<int>(C, -1));
16     queue<pair<int,int>> q;
17     distGrid[startX][startY] = 0;
18     q.push({startX, startY});
19     while (!q.empty()) {
20         auto [x, y] = q.front(); q.pop();
21         for (int d = 0; d < 4; d++) {
22             int nx = x + dx4[d], ny = y + dy4[d];
23             if (dentroDelGrid(nx, ny) && distGrid[nx][ny]
24                 == -1 && grid[nx][ny] != obstaculo) {
25                 distGrid[nx][ny] = distGrid[x][y] + 1;
26                 q.push({nx, ny});
27             }
28         }
29     }
30 }
31 // Contar componentes conexas en grid (ej: contar islas)
32 vector<vector<bool>> visitadoGrid;
33 void dfsGrid(int x, int y, char objetivo) {
34     if (!dentroDelGrid(x, y) || visitadoGrid[x][y] || grid
35         [x][y] != objetivo) return;
36     visitadoGrid[x][y] = true;
37     for (int d = 0; d < 4; d++) dfsGrid(x + dx4[d], y +
38         dy4[d], objetivo);
39 }

```

```

36 int contarIslas(char tierra = '1') {
37     visitadoGrid.assign(R, vector<bool>(C, false));
38     int islas = 0;
39     for (int i = 0; i < R; i++)
40         for (int j = 0; j < C; j++)
41             if (grid[i][j] == tierra && !visitadoGrid[i][j]) {
42                 dfsGrid(i, j, tierra); islas++;
43             }
44     return islas;
45 }

```

Ejemplo de uso

Ejemplo: encontrar la distancia mínima entre dos casillas de un laberinto.

```

1 grid = {
2     "...#",
3     "....",
4     "#..."
5 };
6 R = grid.size();
7 C = grid[0].size();
8
9 bfsGrid(0, 0);
10 cout << distGrid[2][3];

```

5.16 Errores Comunes que Cuestan Penalizaciones

- Olvidar resetear visitado/dist/adj entre casos de prueba (multi-test).
- No verificar límites de un grid/matriz antes de moverse a un vecino.
- Confundir grafo dirigido vs no dirigido al agregar aristas.
- Usar DFS recursivo en grafos gigantes ($n \gtrsim 10^5$) con forma de cadena $\rightarrow$ stack overflow. Usar BFS o DFS iterativo.
- En BFS: marcar visitado al **encolar**, no al desencolar (si no, el mismo nodo entra varias veces a la cola).
- Usar Dijkstra con pesos negativos (da resultados incorrectos): usar Bellman-Ford en ese caso.
- Olvidar que topoSortDFS no detecta ciclos por sí solo: verificar aciclicidad antes, o usar Kahn directamente.
- En 0/1 BFS, confundir el orden de push_front/push_back (0 al frente, 1 al final).

6. Cadenas (Strings)

6.1 KMP (Knuth-Morris-Pratt)

Complejidad de Tiempo: $\mathcal{O}(N + M)$, donde N es el texto y M el patrón.

Complejidad de Espacio: $\mathcal{O}(M)$

Busca todas las ocurrencias de un patrón dentro de un texto sin retroceder en el texto, usando una tabla de fallos (*prefix function*) precalculada sobre el patrón.

```
1 vector<int> build_prefix_function(const string& pat) {
2     int m = pat.size();
3     vector<int> pi(m, 0);
4     for (int i = 1; i < m; i++) {
5         int j = pi[i-1];
6         while (j > 0 && pat[i] != pat[j]) j = pi[j-1];
7         if (pat[i] == pat[j]) j++;
8         pi[i] = j;
9     }
10    return pi;
11 }
12 // Devuelve las posiciones (0-indexadas) en 'texto' donde
13 // inicia 'pat'
14 vector<int> kmp_search(const string& texto, const string&
15 pat) {
16     vector<int> pi = build_prefix_function(pat);
17     vector<int> ocurrencias;
18     int j = 0;
19     for (int i = 0; i < (int)texto.size(); i++) {
20         while (j > 0 && texto[i] != pat[j]) j = pi[j-1];
21         if (texto[i] == pat[j]) j++;
22         if (j == (int)pat.size()) {
23             ocurrencias.push_back(i - j + 1);
24             j = pi[j-1];
25         }
26     }
27     return ocurrencias;
28 }
```

Ejemplo de uso

Ejemplo: encontrar todas las posiciones donde aparece una palabra dentro de un texto.

```
1 string texto = "abracadabra";
2 string patron = "abra";
3
4 vector<int> posiciones = kmp_search(texto, patron);
5 // posiciones = {0, 7}
```

6.2 Z-Function

Complejidad de Tiempo: $\mathcal{O}(N)$

Complejidad de Espacio: $\mathcal{O}(N)$

Para cada posición i , calcula la longitud del prefijo común más largo entre la cadena completa y el sufijo que empieza en i . Útil para *matching* de patrones (concatenando $pat + \# + texto$) y detección de periodicidad.

```
1 vector<int> z_function(const string& s) {
2     int n = s.size();
3     vector<int> z(n, 0);
4     int l = 0, r = 0;
5     for (int i = 1; i < n; i++) {
6         if (i < r) z[i] = min(r - i, z[i - l]);
7         while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
8         if (i + z[i] > r) { l = i; r = i + z[i]; }
9     }
10    return z;
11 }
```

Ejemplo de uso

Ejemplo: comparar rápidamente cada sufijo con el prefijo de una cadena.

```
1 string s = "aaaaab";
2 vector<int> z = z_function(s);
3
4 cout << z[1];
```

6.3 Hashing de Strings (Polinomial)

Complejidad de Tiempo:

- Precomputo: $\mathcal{O}(N)$
- Consulta de hash de subcadena: $\mathcal{O}(1)$

Complejidad de Espacio: $\mathcal{O}(N)$

Permite comparar subcadenas en $\mathcal{O}(1)$ tras el precomputo, útil para detectar duplicados o hacer búsqueda binaria sobre la longitud del prefijo/subcadena común. Se recomienda usar doble hashing (dos módulos) para evitar colisiones adversariales.

```
1 struct StringHash {
2     vector<ll> h, p;
3     ll MOD, BASE;
4     StringHash(const string& s, ll mod = 1e9+7, ll base =
5     131) : MOD(mod), BASE(base) {
6         int n = s.size();
7         h.assign(n + 1, 0); p.assign(n + 1, 1);
8         for (int i = 0; i < n; i++) {
9             h[i+1] = (h[i] * BASE + s[i]) % MOD;
10            p[i+1] = (p[i] * BASE) % MOD;
11        }
12        // Hash del rango [l, r] (0-indexado, inclusivo)
13        ll getHash(int l, int r) {
14            ll res = (h[r+1] - (h[l] * p[r-l+1]) % MOD + MOD *
15            MOD) % MOD;
16            return res;
17        }
18    };
```

Ejemplo de uso

Ejemplo: comparar rápidamente dos subcadenas para detectar si tienen el mismo hash.

```
1 StringHash h("programacion");
2
3 ll a = h.getHash(0, 3);
4 ll b = h.getHash(4, 7);
5
6 cout << (a == b);
```

6.4 Manacher (Palíndromos)

Complejidad de Tiempo: $\mathcal{O}(N)$

Complejidad de Espacio: $\mathcal{O}(N)$

Encuentra el radio del palíndromo más largo centrado en cada posición, manejando de forma unificada los casos de longitud par e impar mediante la inserción de separadores.

```
1 // Retorna, para la cadena transformada t = "#a#b#b#a#",
2 // el radio de palindromo en cada centro
3 vector<int> manacher(const string& s) {
4     string t = "#";
5     for (char c : s) { t += c; t += '#'; }
```

```

5   int n = t.size();
6   vector<int> radio(n, 0);
7   int l = 0, r = -1;
8   for (int i = 0; i < n; i++) {
9       int k = (i > r) ? 1 : min(radio[l + r - i], r - i
10          + 1);
11       while (i - k >= 0 && i + k < n && t[i-k] == t[i+k]
12          ) k++;
13       radio[i] = k--;
14       if (i + k > r) { l = i - k; r = i + k; }
15   }

```

Ejemplo de uso

Ejemplo: encontrar el palíndromo más largo de una palabra.

```

1 string s = "abacaba";
2 vector<int> radio = manacher(s);
3
4 int mejor = 0;
5 for (int r : radio) mejor = max(mejor, r - 1);
6 cout << mejor;

```

6.5 Boyer-Moore (Búsqueda de Patrones)

Complejidad de Tiempo: $\mathcal{O}(N + M)$ en promedio, y en el mejor caso llega a ser **sublineal** ($\mathcal{O}(N/M)$) ya que permite saltar varias posiciones del texto de una vez. Peor caso $\mathcal{O}(N \cdot M)$ sin la heurística de sufijo bueno (poco común en la práctica).

Complejidad de Espacio: $\mathcal{O}(M + \Sigma)$, donde Σ es el tamaño del alfabeto.

A diferencia de KMP (que avanza de izquierda a derecha comparando), Boyer-Moore compara el patrón contra el texto de **derecha a izquierda**, y cuando encuentra un mismatch usa la heurística del “*bad character*” para saltar el patrón varias posiciones de una sola vez, evitando comparaciones innecesarias. Es el algoritmo que usan en la práctica muchos motores de búsqueda de texto (ej. **grep**) por su gran rendimiento en alfabetos grandes (texto natural).

```

1 // Heurística del "bad character": para cada caracter del
2 // alfabeto, guarda
3 // la última posición en la que aparece dentro del patrón
4 vector<int> build_bad_char_table(const string& pat, int
5    alphabetSize = 256) {
6     vector<int> lastOcc(alphabetSize, -1);
7     for (int i = 0; i < (int)pat.size(); i++) lastOcc[(
8         unsigned char)pat[i]] = i;
9     return lastOcc;
10 }
11 // Devuelve las posiciones (0-indexadas) en 'texto' donde
12 // inicia 'pat'
13 vector<int> boyer_moore_search(const string& texto, const
14    string& pat) {
15     int n = texto.size(), m = pat.size();
16     if (m == 0 || m > n) return {};
17     vector<int> badChar = build_bad_char_table(pat);
18     vector<int> ocurrencias;
19     int s = 0; // desplazamiento del patrón respecto al
20     // inicio del texto
21     while (s <= n - m) {
22         int j = m - 1;
23         // Compara de derecha a izquierda mientras los
24         // caracteres coincidan
25         while (j >= 0 && pat[j] == texto[s + j]) j--;
26         if (j < 0) {
27             ocurrencias.push_back(s); // match completo
28             encontrado

```

```

21 // Desplaza usando el caracter siguiente al
22 // patrón (o 1 si no hay mas texto)
23 s += (s + m < n) ? m - badChar[(unsigned char)
24    texto[s + m]] : 1;
25 } else {
26     // Salta según la última ocurrencia del
27     // caracter conflictivo en el patrón
28     int shift = j - badChar[(unsigned char)texto[s
29        + j]];
30     s += max(1, shift); // al menos avanza 1 para
31     // garantizar progreso
32 }
33 }
34 return ocurrencias;
35 }

```

Ejemplo de uso

Ejemplo: buscar una palabra dentro de un texto usando saltos grandes cuando hay coincidencias parciales.

```

1 string texto = "abracadabra";
2 string patron = "cad";
3
4 vector<int> posiciones =
5     boyer_moore_search(texto, patron);

```

Variante: Boyer-Moore Voting Algorithm (Elemento Mayoritario)

Complejidad de Tiempo: $\mathcal{O}(N)$

Complejidad de Espacio: $\mathcal{O}(1)$

No confundir con el algoritmo anterior: comparte el nombre del mismo autor (Robert S. Boyer) pero resuelve un problema distinto — encontrar el elemento que aparece **más de $N/2$ veces** en un arreglo, en una sola pasada y sin memoria extra.

```

1 // Encuentra el candidato a elemento mayoritario (aparece
2 // > n/2 veces)
3 // IMPORTANTE: requiere una segunda pasada para confirmar
4 // que realmente es mayoría,
5 // ya que si no existe tal elemento, el algoritmo igual
6 // retorna un candidato inválido
7 int boyer_moore_voting(const vector<int>& arr) {
8     int candidato = 0, contador = 0;
9     for (int x : arr) {
10         if (contador == 0) candidato = x;
11         contador += (x == candidato) ? 1 : -1;
12     }
13     // Verificación (obligatoria)
14     int freq = count(arr.begin(), arr.end(), candidato);
15     return (freq > (int)arr.size() / 2) ? candidato : -1;
16 }

```

Ejemplo de uso

Ejemplo directo de uso de mayoritario sobre datos pequeños para ver qué resultado produce.

```

1 vector<int> a = {2, 2, 1, 2, 3, 2};
2 cout << mayoritario(a) << '\n';

```

7. Programación Dinámica (DP)

7.1 ¿Cuándo Usar Programación Dinámica?

DP aplica cuando el problema cumple **ambas** condiciones:

- **Subestructura óptima:** la solución óptima del problema completo se puede construir a partir de soluciones óptimas de subproblemas más pequeños.
- **Subproblemas superpuestos:** al resolver recursivamente, los mismos subproblemas se repiten muchas veces (si no se repiten, es simple recursión/divide y vencerás, no DP).

Pistas típicas en el enunciado:

- "Cuenta el número de formas de..." / "¿Cuántas maneras hay de...?"
- "Encuentra el mínimo/máximo costo/valor/longitud para..."
- "¿Es posible lograr...?" (decisión, sí/no) con restricciones acumulativas.
- El problema se puede describir con un estado que depende de "decisiones anteriores" (ej. posición actual, capacidad restante, elementos ya usados).
- Fuerza bruta recursiva sería exponencial, pero el número de **estados distintos** es polinomial.

Cómo diseñar la solución (pasos sugeridos):

1. Define el **estado**: ¿qué información mínima necesito para tomar la siguiente decisión? (ej. índice actual, peso restante, máscara de visitados).
2. Define la **transición**: dado un estado, ¿cómo se llega a los estados siguientes/anteriores?
3. Define el/los **caso(s) base**.
4. Decide el **orden de cálculo**: top-down (memoización, más fácil de escribir) o bottom-up (tabulación, evita stack overflow y suele ser más rápido).
5. Verifica la **complejidad**: (cantidad de estados) $\times$ (costo por transición).

7.2 Plantilla General de DP (Top-Down / Memoización)

Complejidad de Tiempo: (número de estados) $\times$ (costo de la transición).

Complejidad de Espacio: $\mathcal{O}$ (número de estados) para la tabla de memo.

Esqueleto reutilizable: solo hay que cambiar el caso base y la transición según el problema. El ejemplo concreto de la transición cuenta caminos en una grilla moviéndose solo hacia la derecha o hacia abajo (resultado conocido: $\binom{i+j}{i}$).

```
1 // ---- Version con arreglo (mas rapida, requiere conocer
  los limites del estado) ----
2 const int MAXI = 1005, MAXJ = 1005;
3 ll dpGeneral[MAXI][MAXJ];
4 bool visitadoDp[MAXI][MAXJ]; // IMPORTANTE: resetear entre
  casos de prueba (memset o fill)
5
6 ll solve(int i, int j /*, parametros adicionales del
  problema */) {
7     // 1) Caso(s) base
8     if (i < 0 || j < 0) return 0;
9     if (i == 0 && j == 0) return 1;
10    // 2) Memo hit: si ya se calculo este estado,
      devolverlo sin recalcularlo
11    if (visitadoDp[i][j]) return dpGeneral[i][j];
12    visitadoDp[i][j] = true;
13    // 3) Transición: combinar resultados de subproblemas
      mas pequenos
14    ll res = 0;
15    res += solve(i - 1, j);
16    res += solve(i, j - 1);
17    return dpGeneral[i][j] = res % MOD;
18 }
19
20 // ---- Version con map/unordered_map (cuando el estado no
  cabe en un arreglo,
21 // ej. bitmask + nodo, o el rango del estado es muy
  grande/disperso) ----
22 map<pair<int,int>, ll> memoGenerico;
23 ll solveMap(int i, int j) {
24     if (i < 0 || j < 0) return 0;
25     if (i == 0 && j == 0) return 1;
26     auto key = make_pair(i, j);
27     auto it = memoGenerico.find(key);
28     if (it != memoGenerico.end()) return it->second;
29     ll res = solveMap(i - 1, j) + solveMap(i, j - 1);
30     return memoGenerico[key] = res % MOD;
31 }
```

Ejemplo de uso

Ejemplo: contar cuántos caminos existen en una cuadrícula moviéndose solo derecha/abajo.

```
1 memset(visitadoDp, 0, sizeof(visitadoDp));
2
3 cout << solve(3, 4); // cantidad de caminos
```

Tip: si la version recursiva (top-down) hace stack overflow por demasiada profundidad, convertir a *bottom-up* (tabulación): iterar los estados en el orden correcto (de los casos base hacia el estado final) usando bucles for en vez de recursión.

7.3 Fibonacci con Memoización

Ejemplo clásico de subproblemas superpuestos: $F(n) = F(n-1) + F(n-2)$ recalcula los mismos valores exponencialmente sin memoizar. Ver también la sección "Fibonacci Matricial" en el capítulo de Fórmulas Matemáticas Esenciales para la versión $\mathcal{O}(\log n)$.

7.3.1 Versión con Arreglo

Complejidad: $\mathcal{O}(N)$ tiempo y espacio. Ideal cuando $N \lesssim 10^6$ y se conoce el límite de antemano.

```
1 const int MAXN = 1000006;
2 ll memoFib[MAXN];
3 bool calculadoFib[MAXN];
4 ll fibonacci_memo(int n) {
5     if (n <= 1) return n;
6     if (calculadoFib[n]) return memoFib[n];
7     calculadoFib[n] = true;
```

```

8     return memoFib[n] = (fibonacci_memo(n - 1) +
9         fibonacci_memo(n - 2)) % MOD;

```

Ejemplo de uso

Ejemplo: calcular Fibonacci cuando el máximo N se conoce y cabe en un arreglo.

```

1 int n = 40;
2 cout << fibonacci_memo(n);

```

7.3.2 Versión con unordered_map

Complejidad: $\mathcal{O}(N)$ tiempo, sin necesidad de reservar un arreglo de tamaño fijo por adelantado.

```

1 unordered_map<ll, ll> memoFibMap;
2 ll fibonacci_memo_map(ll n) {
3     if (n <= 1) return n;
4     auto it = memoFibMap.find(n);
5     if (it != memoFibMap.end()) return it->second;
6     ll res = (fibonacci_memo_map(n - 1) +
7         fibonacci_memo_map(n - 2)) % MOD;
8     return memoFibMap[n] = res;

```

Ejemplo de uso

Ejemplo directo de uso de fibonacci_memo_map sobre datos pequeños para ver qué resultado produce.

```

1 memoFibMap.clear();
2 cout << fibonacci_memo_map(20) << '\n'; // 6765

```

7.3.3 Fast Doubling con Memoización

Complejidad: $\mathcal{O}(\log N)$. La mejor opción para N gigante (hasta 10^{18}) manteniendo un estilo recursivo/memoizado. Se basa en las identidades:

$$F(2k) = F(k) \cdot (2F(k+1) - F(k)), \quad F(2k+1) = F(k)^2 + F(k+1)^2$$

```

1 unordered_map<ll, pair<ll, ll>> memoFastDouble; // n -> {F(n), F(n+1)}
2 pair<ll, ll> fib_fast_doubling(ll n) {
3     if (n == 0) return {0, 1};
4     auto it = memoFastDouble.find(n);
5     if (it != memoFastDouble.end()) return it->second;
6     auto [fk, fk1] = fib_fast_doubling(n / 2);
7     ll c = (fk * ((2 * fk1 - fk % MOD + MOD) % MOD)) % MOD;
8     // F(2k)
9     ll d = (fk * fk % MOD + fk1 * fk1 % MOD) % MOD;
10    // F(2k+1)
11    pair<ll, ll> res;
12    if (n % 2 == 0) res = {c, d};
13    else res = {d, (c + d) % MOD};
14    return memoFastDouble[n] = res;
15 }
16 ll fibonacci_fast_doubling(ll n) { return
17     fib_fast_doubling(n).first; }

```

Ejemplo de uso

Ejemplo: calcular Fibonacci para valores gigantes en $\mathcal{O}(\log N)$.

```

1 memoFastDouble.clear();
2 cout << fibonacci_fast_doubling(1000000000000LL);

```

7.4 Mochila 0-1 (0-1 Knapsack)

Complejidad de Tiempo: $\mathcal{O}(N \cdot W)$, donde N es la cantidad de elementos y W la capacidad de peso máximo de la mochila.

Complejidad de Espacio: $\mathcal{O}(W)$ mediante optimización con un arreglo unidimensional.

```

1 int knapsack(int W, const vector<int>& peso, const vector<
2     int>& valor, int n) {
3     vector<int> dp(W + 1, 0);
4     for (int i = 0; i < n; i++) {
5         for (int w = W; w >= peso[i]; w--) {
6             dp[w] = max(dp[w], dp[w - peso[i]] + valor[i]);
7         }
8     }
9     return dp[W];

```

Ejemplo de uso

Ejemplo: elegir objetos de mayor valor sin superar el peso máximo de una mochila.

```

1 vector<int> peso = {2, 3, 4};
2 vector<int> valor = {4, 5, 7};
3
4 cout << knapsack(5, peso, valor, peso.size());

```

7.5 Longest Common Subsequence (LCS)

Complejidad de Tiempo: $\mathcal{O}(|S| \cdot |T|)$

Complejidad de Espacio: $\mathcal{O}(|S| \cdot |T|)$

Encuentra la subsecuencia común más larga entre dos strings S y T .

```

1 int lcs(string S, string T) {
2     int n = S.size(), m = T.size();
3     vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
4     for (int i = 1; i <= n; i++) {
5         for (int j = 1; j <= m; j++) {
6             if (S[i-1] == T[j-1]) dp[i][j] = dp[i-1][j-1]
7                 + 1;
8             else dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
9         }
10    }
11    return dp[n][m];

```

Ejemplo de uso

Ejemplo: medir cuánto contenido mantienen en común dos secuencias.

```

1 cout << lcs("ABCBADAB", "BDCABA");

```

7.6 Longest Increasing Subsequence (LIS)

Complejidad de Tiempo: $\mathcal{O}(N \log N)$ mediante búsqueda binaria de reemplazo.

Complejidad de Espacio: $\mathcal{O}(N)$

Calcula la longitud de la subsecuencia estrictamente creciente más larga de un arreglo de tamaño N .

```

1 int lis(const vector<int>& a) {
2     vector<int> tails;
3     for (int x : a) {
4         auto it = lower_bound(tails.begin(), tails.end(),
5             x);
6         if (it == tails.end()) tails.push_back(x);
7         else *it = x;
8     }
9     return tails.size();
}

```

Ejemplo de uso

Ejemplo: encontrar la mayor secuencia de precios que crece estrictamente.

```

1 vector<int> precios = {10, 5, 8, 12, 3, 15};
2 cout << lis(precios);

```

7.7 DP con Máscaras de Bits (Bitmask DP - TSP)

Complejidad de Tiempo: $\mathcal{O}(2^N \cdot N^2)$

Complejidad de Espacio: $\mathcal{O}(2^N \cdot N)$

Resuelve el Problema del Viajero (TSP) encontrando el ciclo hamiltoniano de coste mínimo en un grafo con $N \leq 20$. El estado se representa combinando un bitmask y el nodo actual.

```

1 int n; int dist[20][20]; int dp[1 << 20][20]; //
2     Inicializar en -1
3 int tsp(int mask, int u) {
4     if (mask == (1 << n) - 1) return dist[u][0];
5     if (dp[mask][u] != -1) return dp[mask][u];
6     int ans = 1e9;
7     for (int v = 0; v < n; v++) {
8         if (!(mask & (1 << v))) {
9             ans = min(ans, dist[u][v] + tsp(mask | (1 << v), v));
10        }
11    }
12    return dp[mask][u] = ans;
}

```

Ejemplo de uso

Ejemplo: encontrar la ruta mínima que visita todas las ciudades y regresa al inicio.

```

1 memset(dp, -1, sizeof(dp));
2 cout << tsp(1, 0); // empieza en la ciudad 0

```

7.8 DP en Árboles (Tree DP)

Complejidad de Tiempo: $\mathcal{O}(V)$ (o $\mathcal{O}(V \log V)$ si se combinan estructuras auxiliares por nodo).

Complejidad de Espacio: $\mathcal{O}(V)$

Patrón general para calcular información agregada en árboles mediante DFS post-order: primero se resuelven los hijos, luego se combina el resultado en el padre. El ejemplo calcula el tamaño del subárbol y el conjunto independiente máximo ponderado.

```

1 vector<vector<int>> adj_tree;
2 vector<ll> peso_nodo;
3 ll dpIncluye[200005], dpExcluye[200005]; // Inicializar en 0
4 void tree_dp(int u, int padre) {

```

```

5     dpIncluye[u] = peso_nodo[u];
6     dpExcluye[u] = 0;
7     for (int v : adj_tree[u]) {
8         if (v == padre) continue;
9         tree_dp(v, u);
10        dpIncluye[u] += dpExcluye[v]; // Si tomo u, no
11        // puedo tomar sus hijos
12        dpExcluye[u] += max(dpIncluye[v], dpExcluye[v]);
13    }
14    // Respuesta final: max(dpIncluye[raiz], dpExcluye[raiz])

```

Ejemplo de uso

Ejemplo: escoger un conjunto de empleados sin seleccionar simultáneamente a un jefe y su subordinado directo.

```

1 tree_dp(1, -1);
2 cout << max(dpIncluye[1], dpExcluye[1]);

```

8. Manipulación de Bits (Bitwise)

8.1 Operaciones Básicas y Trucos Fundamentales

Complejidad de Tiempo: $\mathcal{O}(1)$ por operación

Los operadores `&` (AND), `|` (OR), `^` (XOR), `~` (NOT), `<<` y `>>` (shifts) trabajan directamente sobre la representación binaria. Estos son los patrones que más se repiten en cursos.

```
1 int x = 0;
2 // Consultar el bit i (0-indexado desde el menos
  significativo)
3 bool bit_i = (x >> i) & 1;
4 // Prender (set) el bit i
5 x |= (1 << i);
6 // Apagar (clear) el bit i
7 x &= ~(1 << i);
8 // Invertir (toggle) el bit i
9 x ^= (1 << i);
10 // Obtener el bit menos significativo prendido (lowest set
    bit)
11 int lsb = x & (-x);
12 // Apagar el bit menos significativo prendido
13 int sin_lsb = x & (x - 1);
14 // Verificar si x es potencia de 2 (y distinto de 0)
15 bool es_potencia_2 = x > 0 && (x & (x - 1)) == 0;
16 // Redondear hacia arriba a la siguiente potencia de 2
17 ll siguiente_potencia_2(ll n) {
18     ll p = 1;
19     while (p < n) p <<= 1;
20     return p;
21 }
```

Ejemplo de uso

Ejemplo: representar permisos de usuario usando bits.

```
1 int permisos = 0;
2 permisos |= (1 << 2); // activar permiso 2
3 bool puedeEditar = (permisos >> 2) & 1;
```

8.2 Conteo de Bits (Popcount)

Complejidad de Tiempo: $\mathcal{O}(1)$ amortizado con funciones `__builtin` (usan instrucciones nativas del procesador), $\mathcal{O}(\log N)$ con la versión manual.

Contar cuántos bits están prendidos en un número es una operación extremadamente común. GCC ofrece funciones nativas muy rápidas que conviene memorizar.

```
1 int x = 42;
2 int cnt = __builtin_popcount(x); // cuenta bits en
  1 (int, 32 bits)
3 int cnt11 = __builtin_popcount11(x); // version para
  long long (64 bits)
4 int trailing = __builtin_ctz(x); // cantidad de
  ceros al final (trailing zeros)
5 int leading = __builtin_clz(x); // cantidad de
  ceros al inicio (leading zeros)
6 int highBit = 31 - __builtin_clz(x); // indice del bit
  mas significativo prendido
7 // Version manual (por si no se puede usar __builtin, ej.
  lenguajes sin soporte)
8 int popcount_manual(unsigned int x) {
9     int cnt = 0;
10    while (x) { cnt += x & 1; x >>= 1; }
11    return cnt;
12 }
13 // Suma total de bits prendidos en todos los numeros de 1
  a N
14 ll suma_bits_1_a_n(ll n) {
15     ll total = 0;
16     for (int bit = 0; (1LL << bit) <= n; bit++) {
```

```
17         ll ciclo = 1LL << (bit + 1);
18         ll completos = (n + 1) / ciclo;
19         total += completos * (1LL << bit);
20         ll resto = (n + 1) % ciclo;
21         total += max(0LL, resto - (1LL << bit));
22     }
23     return total;
24 }
```

Ejemplo de uso

Ejemplo: contar cuántos permisos están activos en una máscara.

```
1 int permisos = 42;
2 cout << __builtin_popcount(permisos);
```

Tabla de Popcount para Todos los Números en $[0, N]$

Complejidad de Tiempo: $\mathcal{O}(N)$ para construir la tabla completa.

Complejidad de Espacio: $\mathcal{O}(N)$

A diferencia de `suma_bits_1_a_n` (que da un único total acumulado), esta tabla guarda el **popcount individual** de cada número de 0 a N , permitiendo consultas repetidas en $\mathcal{O}(1)$. Se construye con programación dinámica: el popcount de i es igual al popcount de i sin su bit menos significativo, más ese bit. Es la forma estándar de precalcular popcounts masivos sin pagar $\mathcal{O}(\log N)$ por cada consulta individual (o cuando no se puede usar `__builtin_popcount`, ej. jueces que no usan GCC).

```
1 // cnt[i] = cantidad de bits en 1 en la representacion
  binaria de i, para 0 <= i <= n
2 vector<int> popcount_table(int n) {
3     vector<int> cnt(n + 1, 0);
4     for (int i = 1; i <= n; i++) {
5         cnt[i] = cnt[i >> 1] + (i & 1); // i>>1 quita el
        bit menos significativo
    }
6     return cnt;
7 }
8 // Variante equivalente usando i & (i-1) (apaga el lowest
  set bit)
9 vector<int> popcount_table_v2(int n) {
10    vector<int> cnt(n + 1, 0);
11    for (int i = 1; i <= n; i++) {
12        cnt[i] = cnt[i & (i - 1)] + 1;
13    }
14    return cnt;
15 }
16 }
```

Ejemplo de uso

Ejemplo directo de uso de `popcount_table` sobre datos pequeños para ver qué resultado produce.

```
1 auto cnt = popcount_table(20);
2 cout << cnt[13];
```

8.3 Suma y Resta Binaria (Bit a Bit)

Complejidad de Tiempo: $\mathcal{O}(\log N)$

Implementación manual de suma/resta usando solo operadores bitwise, útil para entender cómo el hardware realiza la operación (o cuando se trabaja con aritmética modular sobre

bits, como en criptografía básica). El truco clave: $a \oplus b$ da la suma sin acarreo, y $(a \& b) \ll 1$ da el acarreo a propagar.

```
1 // Suma de dos enteros usando solo operadores bitwise
2 int suma_bitwise(int a, int b) {
3     while (b != 0) {
4         int carry = (a & b) << 1; // acarreo: bits donde
5         // ambos son 1, desplazado
6         a = a ^ b; // suma sin acarreo
7         b = carry;
8     }
9     return a;
10 // Resta usando complemento a dos: a - b = a + (~b + 1)
11 int resta_bitwise(int a, int b) {
12     return suma_bitwise(a, suma_bitwise(~b, 1));
13 }
```

Ejemplo de uso

Ejemplo: realizar una suma sin utilizar directamente el operador $+$.

```
1 cout << suma_bitwise(13, 29);
2 cout << resta_bitwise(29, 13);
```

8.4 Multiplicación Binaria (Russian Peasant / Egyptian Multiplication)

Complejidad de Tiempo: $\mathcal{O}(\log N)$

Multiplica dos números usando solo sumas y desplazamientos de bits, descomponiendo un factor en su representación binaria. Es la misma idea que la exponenciación rápida pero aplicada a la multiplicación, y es clave para hacer **multiplicación modular segura** cuando $a \cdot b$ podría desbordar incluso a long long (ej. módulos cercanos a 10^{18}).

```
1 // Multiplica a*b de forma segura modulo 'mod', evitando
2 // overflow (sin usar __int128)
3 ll mulmod_binario(ll a, ll b, ll mod) {
4     ll result = 0;
5     a %= mod;
6     while (b > 0) {
7         if (b & 1) result = (result + a) % mod; // si el
8         // bit actual de b esta prendido, suma
9         a = (a + a) % mod; // duplica a (equivalente a a
10        // << 1)
11        b >>= 1;
12    }
13    return result;
14 }
15 // Multiplicacion binaria simple sin modulo (solo shifts y
16 // sumas)
17 ll multiplicar_binario(ll a, ll b) {
18     ll result = 0;
19     while (b > 0) {
20         if (b & 1) result += a;
21         a <<= 1;
22         b >>= 1;
23     }
24     return result;
25 }
```

Ejemplo de uso

Ejemplo: multiplicar bajo un módulo muy grande evitando un producto directo peligroso.

```
1 cout << mulmod_binario(123456789LL,
2                        987654321LL,
3                        1000000000000000003LL);
```

8.5 Enumeración de Submáscaras (Submask Enumeration)

Complejidad de Tiempo: $\mathcal{O}(3^N)$ para enumerar todas las submáscaras de todas las máscaras de N bits (no $\mathcal{O}(4^N)$, es una cota más ajustada por el número de pares (mask, submask)).

Técnica clásica para recorrer todos los subconjuntos de un conjunto representado como bitmask, muy usada en DP sobre subconjuntos (*SOS DP* - Sum over Subsets).

```
1 // Recorre todas las submascaras de 'mask' (incluyendo la
2 // mascara vacia y mask mismo)
3 void enumerar_submascaras(int mask) {
4     for (int sub = mask; ; sub = (sub - 1) & mask) {
5         // procesar 'sub' aqui
6         if (sub == 0) break;
7     }
8 }
9 // SOS DP: para cada mascara, suma los valores de todas
10 // sus submascaras
11 // dp[mask] empieza como el valor base de esa mascara
12 // exacta
13 void sos_dp(vector<ll>& dp, int N_BITS) {
14     for (int bit = 0; bit < N_BITS; bit++) {
15         for (int mask = 0; mask < (1 << N_BITS); mask++) {
16             if (mask & (1 << bit)) dp[mask] += dp[mask ^
17             (1 << bit)];
18         }
19     }
20 }
```

Ejemplo de uso

Ejemplo: probar todos los subconjuntos de un conjunto de habilidades representado por una máscara.

```
1 int habilidades = 0b1111;
2
3 for (int sub = habilidades; ; sub = (sub - 1) &
4     habilidades) {
5     cout << sub << '\n';
6     if (sub == 0) break;
7 }
```

8.6 Propiedades y Trucos con XOR

Complejidad de Tiempo: $\mathcal{O}(1)$ a $\mathcal{O}(N)$ según el truco
XOR es asociativo, conmutativo y $x \oplus x = 0$, $x \oplus 0 = x$. Estas propiedades permiten resolver varios problemas clásicos sin memoria extra.

```
1 // Encontrar el UNICO numero que no se repite, cuando
2 // todos los demas aparecen 2 veces
3 int unico_no_repetido(const vector<int>& arr) {
4     int res = 0;
5     for (int x : arr) res ^= x;
6     return res;
7 }
8 // Intercambiar dos variables sin variable auxiliar
9 void swap_xor(int& a, int& b) {
10    if (&a == &b) return; // cuidado: falla si a y b son
11    // la misma direccion
12    a ^= b; b ^= a; a ^= b;
13 }
14 // Prefijos XOR: permite calcular XOR de cualquier rango [
15 // l, r] en  $\mathcal{O}(1)$ 
16 vector<int> prefix_xor(const vector<int>& arr) {
17     vector<int> pre(arr.size() + 1, 0);
18     for (int i = 0; i < (int)arr.size(); i++) pre[i+1] =
19     pre[i] ^ arr[i];
20     return pre;
21 }
22 int xor_rango(const vector<int>& pre, int l, int r) { //
23     // 0-indexado, inclusivo
```

```

19     return pre[r+1] ^ pre[l];
20 }
21 // Encontrar los DOS unicos numeros que no se repiten (los
    demas aparecen 2 veces)
22 pair<int,int> dos_unicos_no_repetidos(const vector<int>&
    arr) {
23     int xorTotal = 0;
24     for (int x : arr) xorTotal ^= x;
25     int diffBit = xorTotal & (-xorTotal); // un bit donde
    los dos unicos difieren
26     int a = 0, b = 0;
27     for (int x : arr) {
28         if (x & diffBit) a ^= x; else b ^= x;
29     }
30     return {a, b};
31 }

```

Ejemplo de uso

Ejemplo: encontrar el número que aparece una sola vez cuando todos los demás aparecen dos veces.

```

1 vector<int> a = {4, 7, 2, 7, 4};
2 cout << unico_no_repetido(a); // 2

```

8.7 std::bitset: Uso y Conversión

Complejidad de Tiempo: $\mathcal{O}(N/64)$ por operación bitwise completa (AND, OR, XOR, shift) sobre todo el bitset, gracias al paralelismo a nivel de palabra del procesador — esto es lo que lo hace mucho más rápido que un `vector<bool>` o un arreglo de bools para operaciones masivas.

`std::bitset<N>` representa una secuencia **fija** de N bits (N debe ser una constante conocida en tiempo de compilación). Es ideal para acelerar DP's booleanos, cribas, y cualquier situación con muchas operaciones bit a bit sobre conjuntos grandes.

```

1 #include <bitset>
2 bitset<32> b1; //
    00000000000000000000000000000000, todo en 0
3 bitset<32> b2(42); // inicializado desde un
    entero
4 bitset<8> b3("10110010"); // inicializado desde un
    string binario
5 // Operaciones basicas
6 b1[3] = 1; // prender el bit 3 (operator[])
7 b1.set(5); // prender el bit 5
8 b1.reset(3); // apagar el bit 3
9 b1.flip(0); // invertir el bit 0
10 bool bit5 = b1.test(5); // consultar el bit 5 (mas
    seguro que operator[])
11 int cantidadPrendidos = b1.count(); // cuenta bits en 1,
    equivalente a popcount
12 bool hayAlguno = b1.any(); // true si algun bit
    esta prendido
13 bool noHayNinguno = b1.none(); // true si todos
    estan en 0
14 bool todosPrendidos = b1.all(); // true si todos
    estan en 1
15 size_t tamano = b1.size(); // tamano fijo N (32
    en este caso)
16 // Operadores bitwise funcionan igual que con enteros,
    pero sobre todo el bitset
17 bitset<32> b4 = b1 & b2;
18 bitset<32> b5 = b1 | b2;
19 bitset<32> b6 = b1 ^ b2;
20 b1 <<= 3; // shift a la izquierda
21 b1 >>= 2; // shift a la derecha

```

Ejemplo de uso

Ejemplo: representar un conjunto pequeño de opciones y hacer operaciones masivas sobre sus bits.

```

1 bitset<8> permisos("10110010");
2
3 cout << permisos.count();
4 permisos.set(0);
5 permisos.reset(4);

```

Conversión a String y a Entero

```

1 bitset<8> b("10110010");
2 // bitset -> string
3 string s = b.to_string(); // "10110010"
4 // bitset -> entero (cuidado con el tamano: to_ulong lanza
    excepcion si no cabe en unsigned long)
5 unsigned long numUL = b.to_ulong(); // para bitsets
    pequenos (<=32 bits usualmente)
6 unsigned long long numULL = b.to_ullong(); // para bitsets
    de hasta 64 bits
7 // entero -> bitset (constructor directo)
8 int x = 178;
9 bitset<8> desdeEntero(x); // 10110010
10 // string -> bitset (constructor directo)
11 string binStr = "10110010";
12 bitset<8> desdeString(binStr);
13 // Tambien se puede construir desde un string con
    caracteres personalizados
14 // (por ejemplo, usando '.'/'#' en vez de '0'/'1', util
    para leer mapas de grilla)
15 string grid = "##..#.#.";
16 bitset<8> desdeGrid(grid, 0, string::npos, '.', '#'); //
    '.' -> 0, '#' -> 1

```

Ejemplo de uso

Consulta directamente el siguiente o anterior elemento en un conjunto ordenado.

```

1 multiset<int> s = {1, 3, 3, 5, 7};
2 auto it = s.lower_bound(4);
3 cout << *it;

```

8.8 Casos de Uso de Bitmasking

Complejidad de Tiempo: varía según el caso, pero comparte el patrón de representar un subconjunto o estado con los bits de un entero.

Además del DP con bitmask ya visto (TSP), estos son los usos más frecuentes en competencia:

Representar Visitados / Subconjuntos en Grafos Pequeños

Cuando $N \leq 20$ -22, un entero puede representar qué nodos ya se visitaron, permitiendo DP sobre subconjuntos como estado.

```

1 // Contar caminos hamiltonianos que empiezan en el nodo 0,
    en un grafo de N nodos
2 int N; bool adj[20][20];
3 ll dp[1 << 20][20]; // dp[mask][u] = caminos que visitan
    exactamente 'mask' y terminan en u
4 ll contar_caminos_hamiltonianos() {
5     memset(dp, 0, sizeof(dp));
6     dp[1][0] = 1; // solo se visito el nodo 0
7     for (int mask = 1; mask < (1 << N); mask++) {
8         for (int u = 0; u < N; u++) {
9             if (!(mask & (1 << u)) || dp[mask][u] == 0)
                continue;
10            for (int v = 0; v < N; v++) {
11                if (adj[u][v] && !(mask & (1 << v))) {
12                    dp[mask | (1 << v)][v] += dp[mask][u];
13                }
14            }
15        }
16    }

```

```

17     ll total = 0;
18     for (int u = 0; u < N; u++) total += dp[(1 << N) - 1][
19         u];
20     return total;

```

Ejemplo de uso

Ejemplo: subset sum acelerado: saber qué sumas pueden conseguirse con un conjunto de números.

```

1 vector<int> pesos = {3, 5, 7};
2 auto alcanzables = subset_sum_alcanzable(pesos);
3
4 cout << alcanzables[10];

```

Problema de Asignación con Bitmask (Assignment Problem)

Asignar N trabajadores a N tareas minimizando el costo, cuando N es pequeño (≤ 20). El bitmask representa qué tareas ya fueron asignadas.

```

1 int N; int costo[20][20];
2 int dpAsig[1 << 20]; // Inicializar en -1
3 // dp(mask) = costo minimo de asignar las tareas prendidas
4 // en 'mask' a los primeros popcount(mask) trabajadores
5 int asignacion_optima(int mask) {
6     int trabajador = __builtin_popcount(mask);
7     if (trabajador == N) return 0;
8     if (dpAsig[mask] != -1) return dpAsig[mask];
9     int mejor = INT_MAX;
10    for (int tarea = 0; tarea < N; tarea++) {
11        if (!(mask & (1 << tarea))) {
12            mejor = min(mejor, costo[trabajador][tarea] +
13                asignacion_optima(mask | (1 << tarea)));
14        }
15    }
16    return dpAsig[mask] = mejor;

```

Ejemplo de uso

Ejemplo directo de uso de dp sobre datos pequeños para ver qué resultado produce.

```

1 int mask = 0b1011;
2 cout << "mask = " << mask << '\n';
3 cout << "bit 1 = " << ((mask >> 1) & 1) << '\n';

```

Subset Sum Acelerado con bitset

Acelera el clásico DP de subset sum de $\mathcal{O}(N \cdot W)$ a $\mathcal{O}(N \cdot W/64)$ representando el conjunto de sumas alcanzables como un bitset, donde el bit i indica si la suma i es alcanzable.

```

1 const int MAXW = 100005;
2 bitset<MAXW> subset_sum_alcanzable(const vector<int>&
3     pesos) {
4     bitset<MAXW> dp;
5     dp[0] = 1; // suma 0 siempre es alcanzable (no
6     // eligiendo nada)
7     for (int w : pesos) {
8         dp |= (dp << w); // para cada suma ya alcanzable s
9         // , ahora s+w tambien lo es
10    }
11    return dp; // dp[x] == 1 si existe un subconjunto que
12    // suma exactamente x

```

Ejemplo de uso

Ejemplo directo de uso de subset_sum_alcanzable sobre datos pequeños para ver qué resultado produce.

```

1 vector<int> pesos = {3, 5, 7};
2 auto alcanzables = subset_sum_alcanzable(pesos);
3 cout << alcanzables[8] << '\n'; // 3 + 5

```

Contar Pares/Tripletas con Propiedades sobre Bits

Cuando se necesita saber, para cada máscara, cuántos elementos del arreglo tienen exactamente esos bits (o son submáscara/supermáscara de ella), se usa un arreglo indexado por máscara en vez de recorrer todo el arreglo cada vez.

```

1 // cnt[mask] = cantidad de elementos del arreglo original
2 // iguales a 'mask'
3 // Tras aplicar SOS DP (ver seccion anterior), cnt[mask]
4 // pasa a significar
5 // "cantidad de elementos que son SUBMASCARA de mask"
6 vector<ll> contar_por_mascara(const vector<int>& arr, int
7     bits) {
8     vector<ll> cnt(1 << bits, 0);
9     for (int x : arr) cnt[x]++;
10    for (int bit = 0; bit < bits; bit++)
11        for (int mask = 0; mask < (1 << bits); mask++)
12            if (mask & (1 << bit)) cnt[mask] += cnt[mask ^
13                (1 << bit)];
14    return cnt;

```

Ejemplo de uso

Ejemplo directo de uso de DP sobre datos pequeños para ver qué resultado produce.

```

1 int mask = 0b1111;
2 for (int sub = mask; ; sub = (sub - 1) & mask) {
3     cout << sub << ' ';
4     if (sub == 0) break;
5 }

```

9. Matemáticas Teóricas

9.1 MCD (Máximo Común Divisor) y MCM (Mínimo Común Múltiplo)

Complejidad de Tiempo: $\mathcal{O}(\log(\min(a, b)))$

Complejidad de Espacio: $\mathcal{O}(1)$ utilizando la función recursiva nativa o el algoritmo de Euclides.

Implementación básica de GCD utilizando recursión y cálculo directo de LCM basándose en la identidad $a \cdot b = \gcd(a, b) \cdot \text{lcm}(a, b)$.

```
1 // Nota: En C++17 existe std::gcd y std::lcm en la
  librería <numeric>
2 long long gcd(long long a, long long b) {
3     return b == 0 ? a : gcd(b, a % b);
4 }
5 long long lcm(long long a, long long b) {
6     if (a == 0 || b == 0) return 0;
7     return (a / gcd(a, b)) * b; // Evita el desbordamiento
  de enteros (overflow)
8 }
```

Ejemplo de uso

Ejemplo: sincronizar dos eventos que se repiten cada a y b segundos.

```
1 ll a = 12, b = 18;
2
3 cout << gcd(a, b); // 6
4 cout << lcm(a, b); // 36
```

9.2 Algoritmo de Euclides Extendido

Complejidad de Tiempo: $\mathcal{O}(\log(\min(a, b)))$

Complejidad de Espacio: $\mathcal{O}(\log(\min(a, b)))$ debido a la pila de recursión.

Halla los coeficientes enteros x, y de la identidad de Bézout para resolver la ecuación lineal $ax + by = \gcd(a, b)$, lo cual sirve también para hallar el inverso multiplicativo modular.

```
1 ll extgcd(ll a, ll b, ll &x, ll &y) {
2     if (b == 0) { x = 1; y = 0; return a; }
3     ll x1, y1;
4     ll d = extgcd(b, a % b, x1, y1);
5     x = y1; y = x1 - y1 * (a / b);
6     return d;
7 }
```

Ejemplo de uso

Ejemplo: encontrar el inverso modular de a cuando $\gcd(a, m) = 1$.

```
1 ll x, y;
2 ll g = extgcd(3, 11, x, y);
3
4 if (g == 1) {
5     x = (x % 11 + 11) % 11;
6     cout << x; // 4
7 }
```

9.3 Criba de Eratóstenes (Sieve)

Complejidad de Tiempo: $\mathcal{O}(N \log \log N)$

Complejidad de Espacio: $\mathcal{O}(N)$

Marca todos los números primos hasta N . La variante mostrada además calcula el menor factor primo (*smallest prime factor*) de cada número, lo cual permite factorizar cualquier valor en $\mathcal{O}(\log N)$.

```
1 const int MAXN = 1000006;
2 vector<int> spf(MAXN); // menor factor primo de cada
  numero
3 vector<int> primos;
4 void sieve() {
5     for (int i = 2; i < MAXN; i++) {
6         if (spf[i] == 0) {
7             spf[i] = i;
8             primos.push_back(i);
9         }
10        for (int p : primos) {
11            if (p > spf[i] || (ll)i * p >= MAXN) break;
12            spf[i * p] = p;
13        }
14    }
15 }
16 // Factoriza n usando el sieve precalculado
17 vector<int> factorizar(int n) {
18     vector<int> factores;
19     while (n > 1) { factores.push_back(spf[n]); n /= spf[n]; }
20     return factores;
21 }
```

Ejemplo de uso

Ejemplo: precalcular primos y después factorizar muchos números rápidamente.

```
1 sieve();
2
3 vector<int> factores = factorizar(360);
4 // 2 2 2 3 3 5
```

9.4 Exponenciación Rápida (Fast/Modular Power)

Complejidad de Tiempo: $\mathcal{O}(\log E)$, donde E es el exponente.

Complejidad de Espacio: $\mathcal{O}(1)$ (versión iterativa)

Calcula $\text{base}^{\text{exp}} \bmod m$ eficientemente mediante exponenciación binaria, evitando el cálculo directo de potencias enormes.

```
1 ll modpow(ll base, ll exp, ll mod) {
2     ll result = 1;
3     base %= mod;
4     while (exp > 0) {
5         if (exp & 1) result = (result * base) % mod;
6         base = (base * base) % mod;
7         exp >>= 1;
8     }
9     return result;
10 }
```

Ejemplo de uso

Ejemplo: calcular una potencia enorme módulo MOD.

```
1 cout << modpow(2, 1000000000LL, 1000000007LL);
```

9.5 Inverso Multiplicativo Modular

Complejidad de Tiempo: $\mathcal{O}(\log MOD)$

Complejidad de Espacio: $\mathcal{O}(1)$

Calcula el inverso de a módulo m (donde m es primo) usando el Pequeño Teorema de Fermat: $a^{-1} \equiv a^{m-2} \pmod{m}$.

```
1 const ll MOD = 1e9 + 7; // debe ser primo
2 ll inv_mod(ll a) {
3     return modpow(a, MOD - 2, MOD);
4 }
```

Ejemplo de uso

Ejemplo: dividir bajo módulo primo usando el inverso modular.

```
1 ll a = 3;
2 ll inv = inv_mod(a);
3
4 cout << inv; // 333333336 para MOD = 1e9+7
```

9.6 Combinatoria: Factoriales y nCr con Módulo

Complejidad de Tiempo: $\mathcal{O}(N \log MOD)$ para el precomputo, $\mathcal{O}(1)$ por consulta de nCr .

Complejidad de Espacio: $\mathcal{O}(N)$

Precalcula factoriales y sus inversos modulares para responder consultas de combinatoria (nCr , nPr) en tiempo constante.

```
1 const int MAXF = 1000006;
2 vector<ll> fact(MAXF), inv_fact(MAXF);
3 void precalc_factoriales() {
4     fact[0] = 1;
5     for (int i = 1; i < MAXF; i++) fact[i] = (fact[i-1] * i) % MOD;
6     inv_fact[MAXF - 1] = inv_mod(fact[MAXF - 1]);
7     for (int i = MAXF - 2; i >= 0; i--) inv_fact[i] = (inv_fact[i+1] * (i+1)) % MOD;
8 }
9 ll nCr(int n, int r) {
10     if (r < 0 || r > n) return 0;
11     return fact[n] * inv_fact[r] % MOD * inv_fact[n-r] % MOD;
12 }
```

Ejemplo de uso

Ejemplo: contar cuántas formas hay de elegir r personas de n .

```
1 precalc_factoriales();
2 cout << nCr(10, 3); // 120
```

9.7 Test de Primalidad Miller-Rabin

Complejidad de Tiempo: $\mathcal{O}(k \log^3 N)$, donde k es el número de rondas de testigos.

Complejidad de Espacio: $\mathcal{O}(1)$

Test de primalidad probabilístico (determinístico para $N < 3.3 \times 10^{14}$ con los testigos mostrados) mucho más rápido que la división por tentativa para números grandes.

```
1 ll mulmod(ll a, ll b, ll m) { return (__int128)a * b % m; }
2 ll modpow128(ll base, ll exp, ll mod) {
3     ll result = 1; base %= mod;
4     while (exp > 0) {
5         if (exp & 1) result = mulmod(result, base, mod);
6         base = mulmod(base, base, mod);
7         exp >>= 1;
8     }
9     return result;
10 }
11 bool miller_rabin(ll n) {
12     if (n < 2) return false;
13     for (ll p : {2,3,5,7,11,13,17,19,23,29,31,37}) {
14         if (n % p == 0) return n == p;
15     }
16     ll d = n - 1; int r = 0;
17     while (d % 2 == 0) { d /= 2; r++; }
18     for (ll a : {2,3,5,7,11,13,17,19,23,29,31,37}) {
19         if (a >= n) continue;
20         ll x = modpow128(a, d, n);
21         if (x == 1 || x == n - 1) continue;
22         bool compuesto = true;
23         for (int i = 0; i < r - 1; i++) {
24             x = mulmod(x, x, n);
25             if (x == n - 1) { compuesto = false; break; }
26         }
27         if (compuesto) return false;
28     }
29     return true;
30 }
```

Ejemplo de uso

Ejemplo: comprobar rápidamente si un número grande es primo.

```
1 ll n = 1000000007LL;
2 cout << (miller_rabin(n) ? "Primo" : "Compuesto");
```

9.8 Teorema Chino del Resto (CRT) — Implementación

Complejidad de Tiempo: $\mathcal{O}(K \log M)$, donde K es la cantidad de congruencias.

Resuelve el sistema $x \equiv r_i \pmod{m_i}$ combinando las congruencias de dos en dos con Euclides extendido. Funciona incluso si los m_i NO son coprimos entre sí (a diferencia de la formulación clásica), siempre que el sistema sea consistente.

```
1 // Combina x ≡ r1 (mod m1) y x ≡ r2 (mod m2) en una sola
2 // congruencia (mod lcm(m1,m2))
3 // Retorna false si el sistema es inconsistente (sin
4 // solucion)
5 bool crt_combinar(ll r1, ll m1, ll r2, ll m2, ll &r, ll &m) {
6     ll x, y;
7     ll g = extgcd(m1, m2, x, y); // reutiliza extgcd de la
8     // seccion anterior (con long long)
9     if ((r2 - r1) % g != 0) return false; // inconsistente
10    ll lcm_ = m1 / g * m2;
11    ll modulo = m2 / g;
12    ll despl = ((r2 - r1) / g % modulo * x % modulo +
13    modulo) % modulo;
14    r = (r1 + m1 * despl) % lcm_;
15    if (r < 0) r += lcm_;
16    m = lcm_;
17    return true;
18 }
19 // Resuelve el sistema completo dado por vectores
20 // paralelos r[] y m[]
21 bool crt(vector<ll>& r, vector<ll>& m, ll &resR, ll &resM) {
22     {
23         resR = 0; resM = 1;
24         for (int i = 0; i < (int)r.size(); i++) {
25             if (!crt_combinar(resR, resM, r[i], m[i], resR,
26             resM)) return false;
27         }
28     }
29 }
```

```

21     return true;
22 }

```

Ejemplo de uso

Ejemplo: encontrar un número que cumpla simultáneamente varios restos.

```

1 vector<ll> r = {2, 3};
2 vector<ll> m = {3, 5};
3 ll resR, resM;
4
5 if (crt(r, m, resR, resM))
6     cout << resR; // 8

```

9.9 Función de Möbius

Complejidad de Tiempo: $\mathcal{O}(N \log \log N)$ para precalcular todos los valores hasta N (criba lineal).

Usada en técnicas de inclusión-exclusión sobre divisores, como contar pares coprimos.

```

1 const int MAXM = 1000006;
2 vector<int> mu(MAXM);
3 void calcular_mobius() {
4     vector<int> spfLocal(MAXM, 0);
5     vector<int> primosLocal;
6     mu[1] = 1;
7     for (int i = 2; i < MAXM; i++) {
8         if (spfLocal[i] == 0) { spfLocal[i] = i;
9             primosLocal.push_back(i); mu[i] = -1; }
10        for (int p : primosLocal) {
11            if (p > spfLocal[i] || (ll)i * p >= MAXM)
12                break;
13            spfLocal[i * p] = p;
14            mu[i * p] = (p == spfLocal[i]) ? 0 : -mu[i];
15        }
16    }
17    // mu(n) = 0 si n tiene un factor primo al cuadrado
18    // mu(n) = (-1)^k si n es producto de k primos distintos

```

Ejemplo de uso

Ejemplo: precalcular $\mu(n)$ para técnicas de inclusión-exclusión sobre divisores.

```

1 calcular_mobius();
2
3 cout << mu[6]; // 1, porque 6 = 2 * 3

```

9.10 Fórmula de Legendre (Exponente de un Primo en $N!$)

Complejidad de Tiempo: $\mathcal{O}(\log_p N)$

Calcula cuántas veces divide el primo p a $N!$, sin calcular el factorial directamente. Útil para saber si $N!$ es divisible por algo, o para calcular $N! \bmod p^k$.

```

1 // Exponente de p en la factorizacion prima de n!
2 ll legendre_exponente(ll n, ll p) {
3     ll cnt = 0;
4     while (n > 0) { n /= p; cnt += n; }
5     return cnt;
6 }

```

Ejemplo de uso

Ejemplo: saber cuántas veces aparece el factor 2 dentro de $100!$.

```

1 cout << legendre_exponente(100, 2);

```

10. Geometría Computacional

10.1 Primitivas Básicas: Punto, Producto Cruz y Punto

Complejidad de Tiempo: $\mathcal{O}(1)$ por operación

Operaciones fundamentales sobre puntos 2D representados como pares (x, y) . El producto cruz determina la orientación relativa de tres puntos (giro izquierda/derecha/colineal).

```
1 struct Point {
2     double x, y;
3     Point operator-(const Point& o) const { return {x - o.x,
4         y - o.y}; }
5 };
6 // Producto cruz de (0->A) x (0->B). Positivo: giro
7 // antihorario (izquierda)
8 double cross(const Point& O, const Point& A, const Point&
9     B) {
10     return (A.x - O.x) * (B.y - O.y) - (A.y - O.y) * (B.x
11         - O.x);
12 }
13 double dot(const Point& O, const Point& A, const Point& B)
14 {
15     return (A.x - O.x) * (B.x - O.x) + (A.y - O.y) * (B.y
16         - O.y);
17 }
18 double dist(const Point& A, const Point& B) {
19     return hypot(A.x - B.x, A.y - B.y);
20 }
```

Ejemplo de uso

Ejemplo: saber si tres puntos forman un giro a la izquierda o a la derecha.

```
1 Point A{0, 0}, B{2, 0}, C{1, 3};
2
3 double area2 = cross(A, B, C);
4 // area2 > 0: C esta a la izquierda de A->B
```

10.2 Envolverte Convexa (Convex Hull - Andrew Monotone Chain)

Complejidad de Tiempo: $\mathcal{O}(N \log N)$, dominado por el ordenamiento de puntos.

Complejidad de Espacio: $\mathcal{O}(N)$

Calcula el polígono convexo más pequeño que contiene a todos los puntos dados, construyendo la cadena inferior y superior por separado.

```
1 vector<Point> convex_hull(vector<Point> pts) {
2     int n = pts.size(), k = 0;
3     if (n < 3) return pts;
4     sort(pts.begin(), pts.end(), [](const Point& a, const
5         Point& b) {
6         return a.x < b.x || (a.x == b.x && a.y < b.y);
7     });
8     vector<Point> hull(2 * n);
9     // Cadena inferior
10    for (int i = 0; i < n; i++) {
11        while (k >= 2 && cross(hull[k-2], hull[k-1], pts[i])
12            <= 0) k--;
13        hull[k++] = pts[i];
14    }
15    // Cadena superior
16    for (int i = n - 2, t = k + 1; i >= 0; i--) {
17        while (k >= t && cross(hull[k-2], hull[k-1], pts[i])
18            <= 0) k--;
19        hull[k++] = pts[i];
20    }
21    hull.resize(k - 1);
22 }
```

```
19     return hull;
20 }
```

Ejemplo de uso

Ejemplo: encontrar el contorno exterior de un conjunto de puntos GPS.

```
1 vector<Point> puntos = {
2     {0,0}, {2,0}, {1,1}, {0,2}, {2,2}
3 };
4
5 vector<Point> borde = convex_hull(puntos);
```

10.3 Área y Punto Dentro de Polígono

Complejidad de Tiempo: $\mathcal{O}(N)$ para ambas operaciones.

Complejidad de Espacio: $\mathcal{O}(1)$

El área se calcula con la fórmula del *shoelace*. La pertenencia de un punto se determina con el algoritmo de *ray casting*, contando cuántas veces un rayo horizontal cruza los bordes del polígono.

```
1 double poly_area(const vector<Point>& poly) {
2     double area = 0;
3     int n = poly.size();
4     for (int i = 0; i < n; i++) {
5         int j = (i + 1) % n;
6         area += poly[i].x * poly[j].y - poly[j].x * poly[i].
7             .y;
8     }
9     return fabs(area) / 2.0;
10 }
11 bool point_in_polygon(const Point& p, const vector<Point>&
12     poly) {
13     int n = poly.size();
14     bool dentro = false;
15     for (int i = 0, j = n - 1; i < n; j = i++) {
16         bool cond = ((poly[i].y > p.y) != (poly[j].y > p.y
17             )) &&
18             (p.x < (poly[j].x - poly[i].x) * (p.y - poly[i].
19                 .y) / (poly[j].y - poly[i].y) + poly[i].x);
20         if (cond) dentro = !dentro;
21     }
22     return dentro;
23 }
```

Ejemplo de uso

Ejemplo: calcular el área de una parcela y comprobar si una coordenada cae dentro.

```
1 vector<Point> parcela = {
2     {0,0}, {4,0}, {4,3}, {0,3}
3 };
4
5 cout << poly_area(parcela);
6 cout << point_in_polygon({2,1}, parcela);
```

10.4 Intersección de Segmentos

Complejidad de Tiempo: $\mathcal{O}(1)$

Determina si dos segmentos de recta se intersectan, cubriendo tanto el caso general (usando orientaciones/producto cruz) como los casos especiales colineales.

```

1 int orientation(const Point& p, const Point& q, const
  Point& r) {
2     double val = cross(p, q, r);
3     if (fabs(val) < 1e-9) return 0; // colineales
4     return (val > 0) ? 1 : 2; // 1: antihorario, 2:
        horario
5 }
6 bool on_segment(const Point& p, const Point& q, const
  Point& r) {
7     return q.x <= max(p.x, r.x) && q.x >= min(p.x, r.x) &&
        q.y <= max(p.y, r.y) && q.y >= min(p.y, r.y);
8 }
9 }
10 bool segments_intersect(const Point& p1, const Point& q1,
  const Point& p2, const Point& q2) {
11     int o1 = orientation(p1, q1, p2), o2 = orientation(p1,
        q1, q2);
12     int o3 = orientation(p2, q2, p1), o4 = orientation(p2,
        q2, q1);
13     if (o1 != o2 && o3 != o4) return true; // caso general
14     if (o1 == 0 && on_segment(p1, p2, q1)) return true;
15     if (o2 == 0 && on_segment(p1, q2, q1)) return true;
16     if (o3 == 0 && on_segment(p2, p1, q2)) return true;
17     if (o4 == 0 && on_segment(p2, q1, q2)) return true;
18     return false;
19 }

```

Ejemplo de uso

Ejemplo: comprobar si dos carreteras se cruzan.

```

1 Point a{0,0}, b{4,4};
2 Point c{0,4}, d{4,0};
3
4 cout << segments_intersect(a, b, c, d);

```

10.5 Punto de Intersección de Dos Rectas

Complejidad de Tiempo: $\mathcal{O}(1)$

Calcula el punto exacto donde se cruzan dos rectas (no segmentos) definidas cada una por dos puntos. Si las rectas son paralelas, el determinante da cero y no hay solución única.

```

1 // Retorna true si existe interseccion unica; el resultado
  se guarda en 'out'
2 bool line_intersection(const Point& p1, const Point& p2,
  const Point& p3, const Point& p4,
3     Point& out) {
4     double a1 = p2.y - p1.y, b1 = p1.x - p2.x;
5     double c1 = a1 * p1.x + b1 * p1.y;
6     double a2 = p4.y - p3.y, b2 = p3.x - p4.x;
7     double c2 = a2 * p3.x + b2 * p3.y;
8     double det = a1 * b2 - a2 * b1;
9     if (fabs(det) < 1e-9) return false; // rectas
        paralelas o coincidentes
10    out.x = (b2 * c1 - b1 * c2) / det;
11    out.y = (a1 * c2 - a2 * c1) / det;
12    return true;
13 }

```

Ejemplo de uso

Ejemplo: encontrar exactamente dónde se cruzan dos trayectorias.

```

1 Point out;
2 bool ok = line_intersection(
3     {0,0}, {4,4},
4     {0,4}, {4,0},
5     out
6 );
7
8 if (ok) cout << out.x << ' ' << out.y;

```

10.6 Distancia Punto-Recta y Punto-Segmento

Complejidad de Tiempo: $\mathcal{O}(1)$

La distancia a la recta usa la proyección normal (magnitud del producto cruz normalizada). La distancia al **segmento** requiere primero verificar si la proyección cae dentro del segmento; si no, la distancia mínima es hacia uno de sus extremos.

```

1 double dist_point_line(const Point& p, const Point& A,
  const Point& B) {
2     return fabs(cross(A, B, p)) / dist(A, B);
3 }
4 double dist_point_segment(const Point& p, const Point& A,
  const Point& B) {
5     if (A.x == B.x && A.y == B.y) return dist(p, A); //
        segmento degenerado
6     double t = dot(A, B, p) / (dist(A, B) * dist(A, B));
7     // parametro de proyeccion
8     if (t < 0) return dist(p, A); // la proyeccion
        cae antes de A
9     if (t > 1) return dist(p, B); // la proyeccion
        cae despues de B
10    Point proj = {A.x + t * (B.x - A.x), A.y + t * (B.y -
        A.y)};
11    return dist(p, proj);

```

Ejemplo de uso

Ejemplo: calcular qué tan cerca está una persona de una carretera.

```

1 Point persona{2, 3};
2 Point A{0,0}, B{5,0};
3
4 cout << dist_point_line(persona, A, B);
5 cout << dist_point_segment(persona, A, B);

```

10.7 Círculos: Pertenencia e Intersección

Complejidad de Tiempo: $\mathcal{O}(1)$

Verifica si un punto está dentro de un círculo y calcula los (hasta 2) puntos de intersección entre dos circunferencias, usando la distancia entre centros para descartar los casos sin solución (muy separados o uno contenido en el otro).

```

1 struct Circle { Point c; double r; };
2 bool point_in_circle(const Point& p, const Circle& circ) {
3     return dist(p, circ.c) <= circ.r + 1e-9;
4 }
5 // Retorna 0, 1 o 2 puntos de interseccion entre dos
  circunferencias
6 vector<Point> circle_circle_intersection(const Circle& c1,
  const Circle& c2) {
7     double d = dist(c1.c, c2.c);
8     if (d > c1.r + c2.r + 1e-9 || d < fabs(c1.r - c2.r) -
        1e-9 || d < 1e-9)
9         return {}; // sin interseccion (muy lejos, uno
        dentro del otro, o concentricos)
10    double a = (c1.r*c1.r - c2.r*c2.r + d*d) / (2*d);
11    double h2 = c1.r*c1.r - a*a;
12    double h = (h2 > 0) ? sqrt(h2) : 0;
13    Point p2 = { c1.c.x + a*(c2.c.x-c1.c.x)/d, c1.c.y + a
        *(c2.c.y-c1.c.y)/d };
14    Point offset = { -(c2.c.y-c1.c.y) * (h/d), (c2.c.x-c1.
        c.x) * (h/d) };
15    Point i1 = { p2.x + offset.x, p2.y + offset.y };
16    Point i2 = { p2.x - offset.x, p2.y - offset.y };
17    if (h < 1e-9) return {i1}; // tangentes: un solo punto
18    return {i1, i2};
19 }

```

Ejemplo de uso

Ejemplo: comprobar si una ubicación está dentro de una zona de cobertura y si dos coberturas se intersectan.

```
1 Circle a{{0,0}, 5};
2 Circle b{{6,0}, 4};
3
4 cout << point_in_circle({3,0}, a);
5
6 auto cortes = circle_circle_intersection(a, b);
```

10.8 Par de Puntos Más Cercano (Closest Pair)

Complejidad de Tiempo: $\mathcal{O}(N \log N)$ mediante divide y vencerás.

Complejidad de Espacio: $\mathcal{O}(N)$

Encuentra la distancia mínima entre dos puntos de un conjunto. Divide el conjunto ordenado por x en dos mitades, resuelve recursivamente y combina revisando solo una franja estrecha alrededor de la línea divisoria.

```
1 inline double dist2(const Point& a, const Point& b) {
2     double dx = a.x - b.x, dy = a.y - b.y;
3     return dx * dx + dy * dy;
4 }
5
6 double closest_pair_rec(vector<Point>& pts, int l, int r)
7 {
8     if (r - l <= 3) {
9         double best2 = 1e100;
10        for (int i = l; i < r; i++)
11            for (int j = i + 1; j < r; j++)
12                best2 = min(best2, dist2(pts[i], pts[j]));
13        sort(pts.begin() + l, pts.begin() + r,
14            [](const Point& a, const Point& b) { return a
15                .y < b.y; });
16        return best2;
17    }
18
19    int mid = (l + r) / 2;
20    double midX = pts[mid].x;
21    double d2 = min(closest_pair_rec(pts, l, mid),
22        closest_pair_rec(pts, mid, r));
23
24    inplace_merge(pts.begin() + l, pts.begin() + mid, pts.
25        begin() + r,
26        [](const Point& a, const Point& b) {
27            return a.y < b.y; });
28
29    vector<Point> strip;
30    strip.reserve(r - l);
31    for (int i = l; i < r; i++) {
32        double dx = pts[i].x - midX;
33        if (dx * dx < d2) strip.push_back(pts[i]);
34    }
35
36    for (int i = 0; i < (int)strip.size(); i++) {
37        for (int j = i + 1; j < (int)strip.size(); j++) {
38            double dy = strip[j].y - strip[i].y;
39            if (dy * dy >= d2) break;
40            d2 = min(d2, dist2(strip[i], strip[j]));
41        }
42    }
43    return d2;
44 }
45
46 double closest_pair(vector<Point> pts) {
47     sort(pts.begin(), pts.end(),
48         [](const Point& a, const Point& b) {
49             return a.x < b.x || (a.x == b.x && a.y < b.y);
50         });
51     return sqrt(closest_pair_rec(pts, 0, (int)pts.size()));
52 }
```

Ejemplo de uso

Ejemplo: encontrar las dos antenas más cercanas entre muchas antenas.

```
1 vector<Point> antenas = {
2     {0,0}, {10,10}, {1,1}, {20,3}
3 };
4
5 cout << closest_pair(antenas);
```

10.9 Rotating Calipers: Diámetro de un Conjunto de Puntos

Complejidad de Tiempo: $\mathcal{O}(N \log N)$ (dominado por el cálculo del convex hull; el barrido de calipers en sí es $\mathcal{O}(N)$). Calcula la distancia máxima entre dos puntos del conjunto. Se apoya en que el par más lejano siempre son dos vértices del *convex hull*, y recorre pares de "puntas" opuestas girando alrededor del polígono.

```
1 double poly_diameter(vector<Point> pts) {
2     vector<Point> hull = convex_hull(pts); // definido en
3     la seccion de Convex Hull
4     int n = hull.size();
5     if (n == 1) return 0;
6     if (n == 2) return dist(hull[0], hull[1]);
7     double best = 0;
8     int j = 1;
9     for (int i = 0; i < n; i++) {
10        int ni = (i + 1) % n;
11        while (fabs(cross(hull[i], hull[ni], hull[(j+1)%n])) >
12            fabs(cross(hull[i], hull[ni], hull[j]))) {
13            j = (j + 1) % n;
14        }
15        best = max({best, dist(hull[i], hull[j]), dist(
16            hull[ni], hull[j])});
17    }
18    return best;
19 }
```

Ejemplo de uso

Ejemplo: encontrar la máxima distancia entre dos puntos de una ciudad.

```
1 vector<Point> puntos = {
2     {0,0}, {4,0}, {4,3}, {0,3}
3 };
4
5 cout << poly_diameter(puntos);
```

10.10 Ordenamiento Angular (Radial Sort) y Convexidad

Complejidad de Tiempo: $\mathcal{O}(N \log N)$ para el ordenamiento; $\mathcal{O}(N)$ para verificar convexidad.

Ordenar puntos por ángulo alrededor de un pivote es la base de algoritmos como Graham Scan. También se muestra cómo verificar si un polígono ya dado es convexo revisando que todos los giros consecutivos tengan el mismo signo.

```
1 // Ordena 'pts' por angulo polar alrededor de 'pivot' (sin
2 // usar atan2, mas preciso)
3 void sort_by_angle(vector<Point>& pts, const Point& pivot)
4 {
5     sort(pts.begin(), pts.end(), [&](const Point& a, const
6         Point& b) {
```

```

4     bool halfA = (a.y > pivot.y) || (a.y == pivot.y &&
5         a.x > pivot.x);
6     bool halfB = (b.y > pivot.y) || (b.y == pivot.y &&
7         b.x > pivot.x);
8     if (halfA != halfB) return halfA > halfB; // parte
9     superior primero
10    double c = cross(pivot, a, b);
11    if (fabs(c) > 1e-9) return c > 0;
12    return dist(pivot, a) < dist(pivot, b); //
13    desempate: mas cercano primero
14    });
15 }
16 bool is_convex_polygon(const vector<Point>& poly) {
17     int n = poly.size();
18     if (n < 3) return false;
19     bool gotPositive = false, gotNegative = false;
20     for (int i = 0; i < n; i++) {
21         double c = cross(poly[i], poly[(i+1)%n], poly[(i
22         +2)%n]);
23         if (c > 1e-9) gotPositive = true;
24         if (c < -1e-9) gotNegative = true;
25         if (gotPositive && gotNegative) return false; //
26         cambia de signo: no convexo
27     }
28     return true;
29 }

```

Ejemplo de uso

Ejemplo: ordenar puntos alrededor de una antena para procesarlos en sentido angular.

```

1 Point centro{0,0};
2 vector<Point> puntos = {
3     {1,0}, {0,1}, {-1,0}, {0,-1}
4 };
5
6 sort_by_angle(puntos, centro);
7 cout << is_convex_polygon(puntos);

```

10.11 Interpolación de Hermite (Cúbica)

Complejidad de Tiempo: $\mathcal{O}(1)$ por evaluación de punto. Construye una curva suave que pasa **exactamente** por un conjunto de puntos dados, especificando además la tangente (dirección/velocidad) en cada uno. A diferencia de interpolar con un polinomio único (que oscila mucho con muchos puntos, fenómeno de Runge), Hermite trabaja *tramo por tramo* entre puntos consecutivos, garantizando continuidad de posición y primera derivada.

```

1 // Funciones base de Hermite cubico (Point definido en la
2 // seccion de Primitivas)
3 double h00(double t) { return 2*t*t*t - 3*t*t + 1; }
4 double h10(double t) { return t*t*t - 2*t*t + t; }
5 double h01(double t) { return -2*t*t*t + 3*t*t; }
6 double h11(double t) { return t*t*t - t*t; }
7 // Evalua el punto en t in [0,1] entre p0 y p1, dadas las
8 // tangentes m0 y m1
9 Point hermite_interp(const Point& p0, const Point& m0,
10                     const Point& p1, const Point& m1,
11                     double t) {
12     double H00 = h00(t), H10 = h10(t), H01 = h01(t), H11 =
13     h11(t);
14     return {
15         H00*p0.x + H10*m0.x + H01*p1.x + H11*m1.x,
16         H00*p0.y + H10*m0.y + H01*p1.y + H11*m1.y
17     };
18 }

```

Ejemplo de uso

Ejemplo: generar una posición suave entre dos puntos con velocidades inicial y final conocidas.

```

1 Point p0{0,0}, p1{10,5};
2 Point m0{5,0}, m1{5,5};
3
4 Point p = hermite_interp(p0, m0, p1, m1, 0.5);

```

Catmull-Rom: Tangentes Automáticas

Cuando no se dan tangentes explícitas, **Catmull-Rom** las calcula automáticamente a partir de los puntos vecinos, produciendo una curva suave que pasa por todos los puntos dados sin necesidad de configurar nada más. Es la forma más práctica de usar Hermite en la mayoría de los problemas (animación de rutas, suavizado de trayectorias).

```

1 // Genera tangentes automaticas: tangente_i = (p_{i+1} -
2 // p_{i-1}) / 2
3 vector<Point> catmull_rom_tangents(const vector<Point>&
4 pts) {
5     int n = pts.size();
6     vector<Point> tan(n);
7     for (int i = 0; i < n; i++) {
8         Point prev = (i == 0) ? pts[i] : pts[i-1];
9         Point next = (i == n-1) ? pts[i] : pts[i+1];
10        tan[i] = { (next.x - prev.x) / 2.0, (next.y - prev
11        .y) / 2.0 };
12    }
13    return tan;
14 }
15 // Evalua un punto en la curva Catmull-Rom completa, con u
16 // en el rango [0, n-1]
17 // (u = 2.3 significa "30% del camino entre el punto 2 y
18 // el punto 3")
19 Point catmull_rom_eval(const vector<Point>& pts, const
20 vector<Point>& tan, double u) {
21     int i = min((int)u, (int)pts.size() - 2);
22     double t = u - i;
23     return hermite_interp(pts[i], tan[i], pts[i+1], tan[i
24 +1], t);
25 }

```

Ejemplo de uso

Ejemplo directo de uso de `catmull_rom_tangents` sobre datos pequeños para ver qué resultado produce.

```

1 Point p0{0, 0}, p1{2, 2}, p2{4, 0};
2 auto tangentes = catmull_rom_tangents(p0, p1, p2);
3 cout << tangentes.first.x << ' ' << tangentes.first.y << '
4     '\n';

```

10.12 Spline Cúbico Natural

Complejidad de Tiempo:

- **Construcción:** $\mathcal{O}(N)$ (resuelve un sistema tridiagonal)
- **Evaluación:** $\mathcal{O}(\log N)$ usando búsqueda binaria para ubicar el tramo (ver capítulo de *Lower Bound y Upper Bound*)

Complejidad de Espacio: $\mathcal{O}(N)$

A diferencia de Hermite (que requiere tangentes dadas o estimadas), el spline cúbico natural resuelve un sistema de ecuaciones para que la curva tenga **segunda derivada continua** en todos los puntos, y además fuerza segunda derivada = 0 en los extremos (condición "natural"). Requiere que los puntos x_i estén **ordenados y sean distintos** (es una función $y = f(x)$, no una curva paramétrica general).

```

1 struct CubicSpline {
2     vector<double> x, a, b, c, d;
3     int n;
4     // xs debe estar ordenado estrictamente creciente
5     CubicSpline(const vector<double>& xs, const vector<
6         double>& ys) {
7         n = xs.size() - 1;
8         x = xs; a = ys;
9         b.assign(n + 1, 0); c.assign(n + 1, 0); d.assign(n
10         + 1, 0);
11         vector<double> h(n), alpha(n + 1, 0);
12         for (int i = 0; i < n; i++) h[i] = x[i+1] - x[i];
13         for (int i = 1; i < n; i++)
14             alpha[i] = 3.0*(a[i+1]-a[i])/h[i] - 3.0*(a[i]-
15             a[i-1])/h[i-1];
16         // Resuelve el sistema tridiagonal para los
17         // coeficientes 'c' (segundas derivadas/6)
18         vector<double> l(n + 1), mu(n + 1), z(n + 1);
19         l[0] = 1; mu[0] = 0; z[0] = 0;
20         for (int i = 1; i < n; i++) {
21             l[i] = 2*(x[i+1] - x[i-1]) - h[i-1]*mu[i-1];
22             mu[i] = h[i] / l[i];
23             z[i] = (alpha[i] - h[i-1]*z[i-1]) / l[i];
24         }
25         l[n] = 1; z[n] = 0; c[n] = 0;
26         for (int j = n - 1; j >= 0; j--) {
27             c[j] = z[j] - mu[j]*c[j+1];
28             b[j] = (a[j+1]-a[j])/h[j] - h[j]*(c[j+1] + 2*c
29             [j])/3.0;
30             d[j] = (c[j+1] - c[j]) / (3.0*h[j]);
31         }
32     }
33     // Evalua el spline en xq (busqueda binaria del tramo
34     // correspondiente)
35     double eval(double xq) {
36         int i = upper_bound(x.begin(), x.end(), xq) - x.
37         begin() - 1;
38         i = max(0, min(i, n - 1));
39         double dx = xq - x[i];
40         return a[i] + b[i]*dx + c[i]*dx*dx + d[i]*dx*dx*dx
41         ; // a + b*dx + c*dx^2 + d*dx^3
42     }
43 };

```

Ejemplo de uso

Ejemplo: interpolar suavemente una serie de mediciones sin forzar cambios bruscos de pendiente.

```

1 CubicSpline spline;
2 spline.build(x, y);
3
4 double estimado = spline.eval(2.5);

```

11. Fórmulas Matemáticas Esenciales (Implementadas)

Referencia rápida de fórmulas que aparecen con frecuencia en el planteamiento de problemas, ya traducidas a código C++ listo para copiar y adaptar.

11.1 Sumas y Series

Complejidad de Tiempo: $\mathcal{O}(1)$ para todas las funciones de esta sección (fórmulas cerradas, sin iterar).

```
1 // Suma de los primeros n naturales: 1+2+...+n = n(n+1)/2
2 ll suma_naturales(ll n) { return n * (n + 1) / 2; }
3
4 // Suma de los primeros n numeros impares: 1+3+5+...+(2n-1) = n^2
5 ll suma_impares(ll n) { return n * n; }
6
7 // Suma de cuadrados: 1^2+2^2+...+n^2 = n(n+1)(2n+1)/6
8 ll suma_cuadrados(ll n) { return n * (n + 1) * (2 * n + 1) / 6; }
9
10 // Suma de cubos: 1^3+2^3+...+n^3 = (n(n+1)/2)^2
11 ll suma_cubos(ll n) { ll s = suma_naturales(n); return s * s; }
12
13 // Suma de una progresion aritmetica con n terminos,
14 // primer termino a1, razon d
15 ll suma_progresion_aritmetica(ll a1, ll d, ll n) {
16     ll an = a1 + (n - 1) * d; // ultimo termino
17     return n * (a1 + an) / 2;
18 }
19
20 // Suma de una progresion geometrica finita: a1 + a1*r + ... + a1*r^(n-1), r != 1
21 // (usar modpow/inv_mod si se necesita modulo, ver capitulo de Matematicas Teoricas)
22 double suma_geometrica(double a1, double r, ll n) {
23     if (r == 1) return a1 * n;
24     return a1 * (pow(r, n) - 1) / (r - 1);
25 }
26
27 // Suma geometrica infinita, |r| < 1
28 double suma_geometrica_infinita(double a1, double r) {
29     return a1 / (1 - r);
30 }
31
32 // Aproximacion de la serie armonica: 1 + 1/2 + 1/3 + ... + 1/n
33 // Util para estimar complejidad O(n log n) de cribas/ divisores sin sumar termino a termino
34 const double EULER_MASCHERONI = 0.5772156649;
35 double serie_armonica_aprox(ll n) {
36     return log((double)n) + EULER_MASCHERONI;
37 }
```

Ejemplo de uso

Ejemplo: calcular una suma hasta N sin recorrer todos los números.

```
1 ll n = 1000000;
2 cout << suma_naturales(n);
3 cout << suma_impares(n);
```

11.2 Combinatoria

Complejidad de Tiempo: $\mathcal{O}(1)$ con nCr precalculado (ver fact/inv_fact en el capítulo de Matemáticas Teóricas); $\mathcal{O}(N)$ para Catalan/derangements si se calculan en tabla.

```
1 // Permutaciones sin repeticion: P(n,r) = n! / (n-r)!
2 // Requiere fact[] e inv_fact[] precalculados (ver Combinatoria en Matematicas Teoricas)
3 ll nPr(int n, int r) {
4     if (r < 0 || r > n) return 0;
5     return fact[n] * inv_fact[n - r] % MOD;
6 }
7 // Combinaciones: ya definido como nCr(n, r) en el capitulo de Matematicas Teoricas
8 // (se repite aqui la firma como referencia): ll nCr(int n, int r);
9
10 // Total de subconjuntos de un conjunto de n elementos: 2^n
11 ll total_subconjuntos(int n) { return modpow(2, n, MOD); }
12
13 // Combinaciones con repeticion (multiconjuntos de tamano r tomados de n tipos)
14 ll combinaciones_con_repeticion(int n, int r) { return nCr(n + r - 1, r); }
15
16 // Permutaciones con repeticion: n! / (n1! * n2! * ... * nk!)
17 ll permutaciones_con_repeticion(int n, const vector<int>& grupos) {
18     ll res = fact[n];
19     for (int ni : grupos) res = res * inv_fact[ni] % MOD;
20     return res;
21 }
22
23 // Identidad de Vandermonde: C(m+n, r) = sum_{k=0}^r C(m, k) * C(n, r-k)
24 // Sirve como checksum/derivacion; rara vez se usa directamente en codigo,
25 // pero se implementa por si se necesita validar o calcular de forma indirecta
26 ll vandermonde(int m, int n, int r) {
27     ll res = 0;
28     for (int k = 0; k <= r; k++) res = (res + nCr(m, k) * nCr(n, r - k)) % MOD;
29     return res;
30 }
31
32 // Identidad del palo de hockey: sum_{i=r}^n C(i, r) = C(n+1, r+1)
33 // (evita sumar n-r terminos; calcular directo con nCr(n+1, r+1))
34 ll hockey_stick(int n, int r) { return nCr(n + 1, r + 1); }
35
36 // Numero de derangements (permutaciones sin puntos fijos), via recurrencia
37 // D_n = (n-1) * (D_{n-1} + D_{n-2}), D_0 = 1, D_1 = 0
38 vector<ll> tabla_derangements(int n) {
39     vector<ll> D(n + 1);
40     D[0] = 1; if (n >= 1) D[1] = 0;
41     for (int i = 2; i <= n; i++)
42         D[i] = (ll)(i - 1) * ((D[i-1] + D[i-2]) % MOD) % MOD;
43     return D;
44 }
45
46 // Números de Catalan. La recurrencia cuadrática es correcta, pero innecesaria
47 // cuando n < MOD: C_i = C_{i-1} * (4i-2) / (i+1).
48 vector<ll> tabla_catalan(int n) {
49     vector<ll> C(n + 1, 0);
50     C[0] = 1;
51     for (int i = 1; i <= n; i++) {
52         // Requiere i + 1 < MOD para usar inv_mod.
53         C[i] = C[i - 1] * (4LL * i - 2) % MOD * inv_mod(i + 1) % MOD;
54     }
55     return C;
56 }
57 // Version O(1) por consulta usando factoriales precalculados:
58 ll catalan_directo(int n) {
59     return (nCr(2 * n, n) - nCr(2 * n, n + 1) % MOD + MOD) % MOD;
60 }
```

Ejemplo de uso

Ejemplo: contar permutaciones y combinaciones usando factoriales precalculados.

```
1 precalc_factoriales();
2
3 cout << nPr(10, 3);
4 cout << nCr(10, 3);
```

11.3 Geometría

Complejidad de Tiempo: $\mathcal{O}(1)$ por fórmula (ver también el capítulo de Geometría Computacional para Point, cross, dist, convex hull, etc.).

```
1 // Area de triangulo con formula de Heron (lados a, b, c)
2 double area_triangulo_heron(double a, double b, double c)
3 {
4     double s = (a + b + c) / 2.0;
5     return sqrt(s * (s - a) * (s - b) * (s - c));
6 }
7 // Area de triangulo por coordenadas (shoelace de 3 puntos)
8 double area_triangulo_coords(double x1, double y1, double
9     x2, double y2, double x3, double y3) {
10     return fabs(x1*(y2-y3) + x2*(y3-y1) + x3*(y1-y2)) /
11         2.0;
12 }
13 // Area de trapecio (bases b1, b2, altura h)
14 double area_trapecio(double b1, double b2, double h) {
15     return (b1 + b2) * h / 2.0;
16 }
17 // Area de poligono regular de n lados de longitud l
18 double area_poligono_regular(int n, double l) {
19     return (n * l * l) / (4.0 * tan(M_PI / n));
20 }
21 // Teorema de Pitagoras: dado a y b, obtener hipotenusa; o
22 // despejar un cateto
23 double hipotenusa(double a, double b) { return sqrt(a*a +
24     b*b); }
25 double cateto(double hip, double otroCateto) { return sqrt
26     (hip*hip - otroCateto*otroCateto); }
27 // Ley de cosenos: c^2 = a^2 + b^2 - 2ab*cos(C), con C en
28 // radianes
29 double ley_cosenos(double a, double b, double anguloC_rad)
30 {
31     return sqrt(a*a + b*b - 2*a*b*cos(anguloC_rad));
32 }
33 // Despejar el angulo C conocidos los 3 lados
34 double angulo_desde_lados(double a, double b, double c) {
35     return acos((a*a + b*b - c*c) / (2*a*b)); // en
36     radianes
37 }
38 // Ley de senos: a/sin(A) = b/sin(B) = c/sin(C). Dado un
39 // lado y su angulo opuesto,
40 // mas otro angulo, obtener el lado faltante.
41 double ley_senos_lado(double ladoConocido, double
42     anguloOpuestoConocido_rad, double
43     anguloOpuestoBuscado_rad) {
44     return ladoConocido * sin(anguloOpuestoBuscado_rad) /
45         sin(anguloOpuestoConocido_rad);
46 }
47 // Distancia entre dos puntos (ver tambien dist(Point,
48 // Point) en Geometria Computacional)
49 double distancia_2d(double x1, double y1, double x2,
50     double y2) {
51     return sqrt((x2-x1)*(x2-x1) + (y2-y1)*(y2-y1));
52 }
53 // Circulo: perimetro y area
54 double circulo_perimetro(double r) { return 2.0 * M_PI * r
55     ; }
56 double circulo_area(double r) { return M_PI * r * r; }
57 // Esfera: volumen y area superficial
```

```
49 double esfera_volumen(double r) { return (4.0/3.0) * M_PI
50     * r * r * r; }
51 double esfera_area(double r) { return 4.0 * M_PI * r * r;
52     }
53 // Cilindro: volumen y area total (2 tapas + lateral)
54 double cilindro_volumen(double r, double h) { return M_PI
55     * r * r * h; }
56 double cilindro_area_total(double r, double h) { return 2*
57     M_PI*r*h + 2*M_PI*r*r; }
58 // Cono: volumen y area lateral (g = generatriz = sqrt(r^2
59     + h^2))
60 double cono_volumen(double r, double h) { return (M_PI * r
61     * r * h) / 3.0; }
62 double cono_area_lateral(double r, double h) {
63     double g = sqrt(r*r + h*h);
64     return M_PI * r * g;
65 }
66 // Teorema de Pick: Area = I + B/2 - 1
67 // I = puntos enteros interiores, B = puntos enteros en el
68 // borde
69 double pick_area(ll puntosInteriores, ll puntosBorde) {
70     return puntosInteriores + puntosBorde / 2.0 - 1.0;
71 }
72 // Puntos enteros estrictamente entre dos vertices de
73 // coordenadas enteras (para calcular B)
74 ll puntos_enteros_en_segmento(ll x1, ll y1, ll x2, ll y2)
75 {
76     return __gcd(llabs(x2 - x1), llabs(y2 - y1)) - 1;
77 }
78 // Dado el area (ej. por shoelace) y el borde B, despejar
79 // los puntos interiores I
80 ll pick_puntos_interiores(double area, ll puntosBorde) {
81     return (ll)llround(area - puntosBorde / 2.0 + 1.0);
82 }
```

Ejemplo de uso

Ejemplo: calcular el área de un triángulo a partir de sus lados o coordenadas.

```
1 cout << area_triangulo_heron(3, 4, 5);
2
3 cout << area_triangulo_coords(
4     {0,0}, {4,0}, {0,3}
5 );
```

11.4 Probabilidad y Recurrencias

Complejidad de Tiempo: $\mathcal{O}(1)$ para las fórmulas puntuales; $\mathcal{O}(\log n)$ para Fibonacci vía exponenciación de matrices.

```
1 // Esperanza de la suma: E[X+Y] = E[X] + E[Y] (SIEMPRE se
2 // cumple, sin importar independencia)
3 double esperanza_suma(const vector<double>&
4     esperanzasIndividuales) {
5     double total = 0;
6     for (double e : esperanzasIndividuales) total += e;
7     return total;
8 }
9 // Varianza: Var(X) = E[X^2] - (E[X])^2
10 double varianza(const vector<double>& valores, const
11     vector<double>& probabilidades) {
12     double EX = 0, EX2 = 0;
13     for (int i = 0; i < (int)valores.size(); i++) {
14         EX += valores[i] * probabilidades[i];
15         EX2 += valores[i] * valores[i] * probabilidades[i];
16     }
17     return EX2 - EX * EX;
18 }
19 // Teorema de Bayes: P(A|B) = P(B|A) * P(A) / P(B)
20 double bayes(double pB_dado_A, double pA, double pB) {
21     return (pB_dado_A * pA) / pB;
22 }
```

```

22 // Probabilidad total: P(A) = sum_i P(A|Bi) * P(Bi)
23 // pA_dado_B[i] y pB[i] son vectores paralelos, uno por
24 // cada evento Bi del particionamiento
25 double probabilidad_total(const vector<double>& pA_dado_B,
26   const vector<double>& pB) {
27     double res = 0;
28     for (int i = 0; i < (int)pA_dado_B.size(); i++) res +=
29       pA_dado_B[i] * pB[i];
30     return res;
31 }
32 // Inclusion-exclusion general para n conjuntos, dado l
33 // interseccion de cualquier subconjunto S|
34 // como funcion interseccion(S) donde S es una mascara de
35 // bits (que conjuntos intervienen)
36 // |Union| = sum sobre todas las submascaras no vacias,
37 // con signo (-1)^(|S|+1)
38 template <typename F>
39 ll inclusion_exclusion(int n, F interseccion) {
40     ll total = 0;
41     for (int mask = 1; mask < (1 << n); mask++) {
42         int bits = __builtin_popcount(mask);
43         ll inter = interseccion(mask);
44         total += (bits % 2 == 1) ? inter : -inter;
45     }
46     return total;
47 }
48 // Caso base con 2 conjuntos: |A U B| = |A| + |B| - |A ^ B|
49 ll union_dos_conjuntos(ll tamA, ll tamB, ll
50   tamInterseccion) {
51     return tamA + tamB - tamInterseccion;
52 }
53 // Fibonacci: formula cerrada (Binet) -- ojo con precision
54 // para n grande, mejor usar matrices
55 double fibonacci_binet(int n) {
56     const double phi = (1.0 + sqrt(5.0)) / 2.0;
57     const double psi = (1.0 - sqrt(5.0)) / 2.0;
58     return round((pow(phi, n) - pow(psi, n)) / sqrt(5.0));
59 }
60 // Fibonacci mod M en O(log n) via exponenciacion de
61 // matrices 2x2
62 // [[F(n+1), F(n)], [F(n), F(n-1)]] = [[1,1],[1,0]]^n
63 struct Mat2 { ll a, b, c, d; };
64 Mat2 matmul(const Mat2& X, const Mat2& Y, ll mod) {
65     return {
66         (X.a*Y.a + X.b*Y.c) % mod, (X.a*Y.b + X.b*Y.d) %
67         mod,
68         (X.c*Y.a + X.d*Y.c) % mod, (X.c*Y.b + X.d*Y.d) %
69         mod
70     };
71 }
72 ll fibonacci_matriz(ll n, ll mod) {
73     if (n == 0) return 0;
74     Mat2 result = {1, 0, 0, 1}; // identidad
75     Mat2 base = {1, 1, 1, 0};
76     ll e = n;
77     while (e > 0) {
78         if (e & 1) result = matmul(result, base, mod);
79         base = matmul(base, base, mod);
80         e >>= 1;
81     }
82     return result.b; // F(n) queda en la posicion [0][1]
83 }

```

Ejemplo de uso

Ejemplo: calcular la esperanza y varianza de la suma de variables aleatorias independientes.

```

1 cout << esperanza_suma(valores);
2 cout << varianza(valores);

```

11.5 Teorema Maestro (guía de referencia, no se codea)

Para recurrencias tipo $T(n) = a \cdot T(n/b) + f(n)$ con $f(n) = O(n^d)$, esto no se implementa (es una herramienta de análisis en papel), pero conviene tenerlo a mano para estimar la complejidad de un divide y vencerás antes de programarlo:

```

1 // Guia rapida (comparar d contra log_b(a)):
2 //   d < log_b(a) -> T(n) = O(n^(log_b a))
3 //   d == log_b(a) -> T(n) = O(n^d * log n)
4 //   d > log_b(a) -> T(n) = O(n^d)
5 // Ejemplo: Merge Sort -> T(n) = 2T(n/2) + O(n) => a=2,
6 //           b=2, d=1, log_2(2)=1 => O(n log n)
7 // Ejemplo: Karatsuba -> T(n) = 3T(n/2) + O(n) => a=3,
8 //           b=2, d=1, log_2(3)=1.58 => O(n^1.58)
9 double log_b(double a, double b) { return log(a) / log(b);
10 } // ayuda a comparar d vs log_b(a)

```

Ejemplo de uso

Ejemplo directo de uso de rapida sobre datos pequeños para ver qué resultado produce.

```

1 // El Teorema Maestro no es una funcion para ejecutar:
2 // se usa para clasificar recurrencias como
3 // T(n) = aT(n/b) + f(n).
4 // Ejemplo: T(n)=2T(n/2)+O(n) -> O(n log n).

```

12. Casos de Estudio: Problemas Resueltos

12.1 Matrioshka

```
1 #include <bits/stdc++.h>
2 #include <sstream>
3 #include <string>
4 using namespace std;
5 int main() {
6     string a;
7     while(getline(cin, a)) {
8         stringstream ss(a);
9         stack<int> mat;
10        stack<int> sumcajas;
11        bool rot = false;
12        int c;
13        while(ss >> c) {
14            if(c < 0) {
15                mat.push(-c);
16                sumcajas.push(0);
17            }
18            else if(!rot) {
19                if(mat.empty() || mat.top() != c) {
20                    rot = true;
21                }
22                else {
23                    mat.pop();
24                    int hijos = sumcajas.top();
25                    sumcajas.pop();
26                    if(hijos >= c) {
27                        rot = true;
28                    }
29                    else if(!sumcajas.empty()) {
30                        int padre = sumcajas.top();
31                        sumcajas.pop();
32                        sumcajas.push(padre + c);
33                    }
34                }
35            }
36        }
37        if(!mat.empty()) rot = true;
38        if(rot) cout << ":-( Try again." << endl;
39        else cout << ":-) Matrioshka!" << endl;
40    }
41 }
```

Ejemplo de uso

Ejemplo: resolver el problema completo: comprobar si una secuencia de tamaños puede formar muñecas anidadas válidas.

```
1 int n;
2 cin >> n;
3
4 vector<int> cajas(n);
5 for (int &x : cajas) cin >> x;
6
7 // El programa de la seccion aplica la pila
8 // para verificar si todas las Matrioshkas son validas.
```